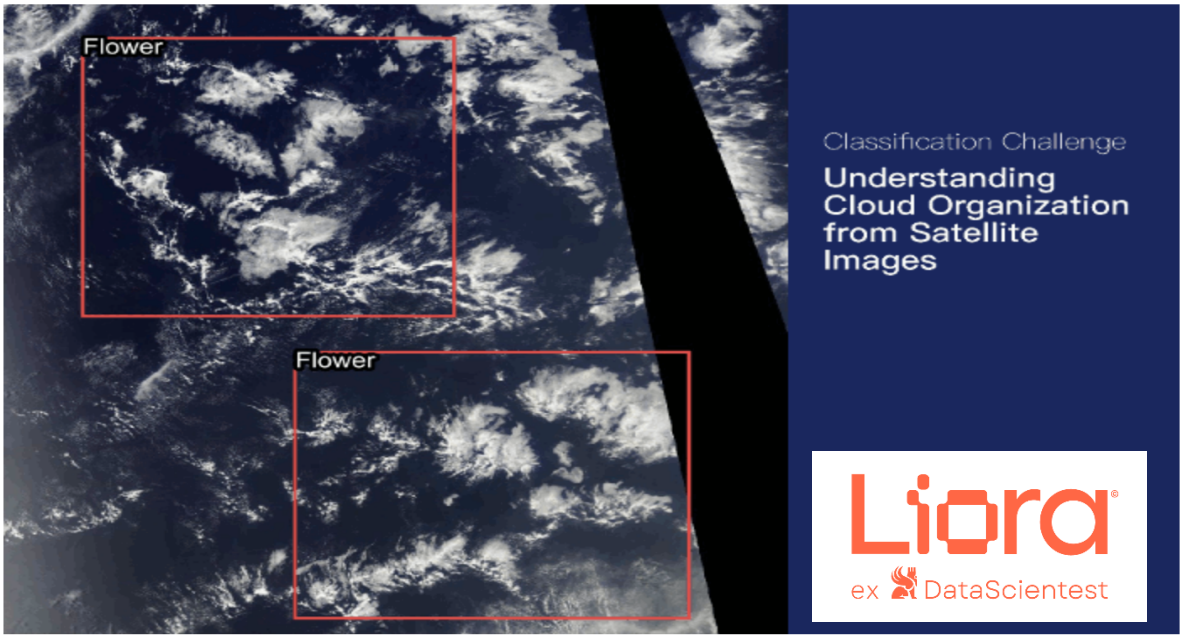


Project: Segmentation of Cloud Images

Authors: Fatemeh Tayebi Naeempour, Mahsa Nakhostinrouhi, Uwe Rosner and Christian J. Schäfer

Promotion: Bootcamp June 2026

Supervision: Yohan Cohen (Liora)

Project Presentation: 22 September 2026

Table of Content

Table of Figures **5**

Introduction **7**

Problem and Objective 7

Course of Analysis 7

1. Exploratory Data Analysis and Data Preprocessing **8**

1.1. Exploratory Data Analysis 8

1.1.1. Data Set Overview 8

1.1.2. Dataset Structure 9

1.1.3. Class Distribution 10

1.1.4. Mask Availability and Cloud Categories per Image 11

1.1.5. Sensor Gap Analysis 12

1.2. Data Preparation 13

1.2.1. RLE Decoding 13

1.2.2. Multi-channel Mask Creation 13

1.2.3. Image and Ground Truth Mask Visualization 13

1.2.4. Image Resizing 14

1.2.5. Normalization 15

1.2.6. Data Augmentation 15

Albumentations Library 15

1.3. Data Preparation Results 18

1.3.1. Augmented Images 18

1.3.2. Train/Validation Split 18

1.3.3. Data Loaders 19

1.3.4. Batch Size 19

1.3.5. Data Format 19

3. Cloud Segmentation Models 1: Cloud Segmentation Using U-Net **20**

3.1. Executive Summary 20

3.1.1. Description of U-Net Method 20

3.1.2. Common Applications of U-Net 21

3.1.3. Rationale for using the U-Net for this Cloud Project 21

3.1.1. Application of U-Net in this Project 22

3.1.2. Results and Interpretation 23

3.1.3. Final validation metrics 24

3.1.4. Predictions on unseen test images 25
3.1.1. Results Still Needed for a Complete Final Model Assessment 26
3.1.2. Recommended Next Improvements 27
4. Cloud Segmentation Models 2: Cloud Segmentation Using U-Net with ResNet50.28
4.1. Executive Summary 28
4.2. Theoretical Background 28
4.2. Application of this U-Net model 29
Architecture as Instantiated 29
4.2.1. Loss Function 31
4.2.2. Optimizer and Learning Rate 31
4.2.3. Training Configuration 31
4.2.4. Training Results 32
4.2.1. Final Validation Metrics 33
4.2.2. Prediction Visualization 34
4.2.3. Test Set Prediction 35
4.4. Conclusion 37
4.2.1. Application of PSPNet in this Project 39
4.2.2. Results and Interpretations 41
4.2.3. Final validation metrics 42
4.2.4. Predictions on unseen test images 43
6.2 Validation Performance Comparison 46
6.3 Comparison of Model Predictions 47
6.4 Final Model Comparison 48
6.5. Recommended Model Selection 48
Summary and Conclusion 50

Table of Figures

Figure 1: Initial dataframe of the training dataset 8

Figure 2: Dataset structure after separating image identifiers and cloud categories 8

Figure 1: Distribution of available segmentation masks by cloud type.....9

Figure 2: Number of Cloud Types per Image 10

Figure 3: Example of sensor gap detection 11

Figure 4: Satellite image 002be4f.jpg with ground truth segmentation masks 13

Figure 7: Python code to import the albumentations library as A.....16

Figure 8: Python code used to train the dataset using CloudSegmentationDataset 17

Figure 5: Original image with masks compared to an augmented version 17

Figure 6: Core U-Net components and their role in the cloud project comparison.....20

Figure 7: Project requirements and reasons for using U-Net.....21

Figure 8: U-Net architecture used in the four-class cloud-segmentation model 21

Figure 9: Training and validation loss and Dice coefficient across 15 displayed epochs. 23

Figure 10: Model 1 Final validation metrics. 23

Figure 11: Model 1 Interpretation of training and validation behavior. 24

Figure 12: Model 1 predicted cloud pattern masks for randomly selected test images 25

Figure 13: Model 1 Recommended next improvements. 26

Figure 14: Model 2 Detailed architecture of the U-Net with ResNet50.....28

Figure 15: Model 2 Training and Validation Loss across 25 Epochs 30

Figure 16: Model 2 Cloud Class, Dice Score and IoU comparison.....31

Figure 17: Model 2 Validation Performance of the U-Net with ResNet50.....31

Figure 18: Model 2 comparison of Satellite Images, Ground Truth Masks, and Predictions ..32

Figure 19: Model 2 Example Test Set Predictions 34

Figure 20: Model 3 Project overview. 35

Figure 21: Model 3 Core PSPNet components and their role in this project. 36

Figure 22: Model 3 instantiated ResNet-34 encoder PSPNet architecture 37

Figure 23: Model 3 training configuration 37

Figure 24: Model 3 training and validation loss (BCE + Dice) over 25 epochs 38

Figure 25: Model 3 training and validation Dice score over 25 epochs 38

Figure 26: Model 3 final validation metrics of the saved model.....39

Figure 27: Model 3 Per-class validation scores 39
Figure 28: Model 3 segmentation map for test image 0035ae9.jpg 40
Figure 29: Model 3 segmentation map for test image 004f759.jpg.....40
Figure 30: Macro Dice per cloud type, PSPNet vs. U-Net (ResNet-50) comparison.....42
Figure 31: Illustrative comparison of model prediction results 43
Figure 32: Final comparison of best validation dice results 43

Introduction

Problem and Objective

Clouds play a crucial role in the Earth's climate system by influencing the energy balance and weather patterns. However, their complex structures and spatial organization make them difficult to represent accurately in climate models. Understanding different cloud organization patterns can help scientists improve climate predictions and reduce uncertainties in future climate projections.

In this project, we aimed to use satellite images and deep learning techniques to segment different cloud organization patterns. By accurately detecting cloud classes and their spatial distribution, this work can contribute to a better understanding of cloud organization and atmospheric processes.

Course of Analysis

We used an existing dataset of labelled satellite images to develop and evaluate deep learning models for cloud segmentation. Chapter 2 introduces the dataset, presents the findings from the exploratory data analysis, and describes the data preprocessing steps.

We followed an iterative modelling approach and evaluated four model configurations. Chapter 3 introduces the compact U-Net baseline. Chapter 4 presents the U-Net models with ResNet34 and ResNet50 encoders, while Chapter 5 introduces the PSPNet model with a ResNet34 encoder. Chapter 6 compares the validation performance and predictions of the different model configurations and discusses the final model selection.

The final Chapter 7 summarizes the main findings of the project, discusses the overall results, and presents the conclusion.

1. Exploratory Data Analysis and Data Preprocessing

1.1. Exploratory Data Analysis

1.1.1. Data Set Overview

The dataset used in this project is the Kaggle *Understanding Clouds from Satellite Images* dataset. It consists of satellite images and corresponding cloud segmentation annotations for four cloud categories: **Fish, Flower, Gravel, and Sugar**.

The dataset contains **5,546 unique satellite images** and **22,184 annotation records**, with **5,546 records for each cloud category**. The segmentation annotations are provided in **Run-Length Encoding (RLE)** format through the *EncodedPixels* column of the training CSV file.

The original dataset images have a resolution of **1400 × 2100 pixels**. For this project, the images were processed and resized to **576 × 384 pixels**, corresponding to a NumPy shape of **384 × 576 × 3**.

The dataset is provided through several files:

- *train.csv*: contains the RLE-encoded segmentation masks for each image and cloud-label pair in the training set.
- *train_images.zip*: contains the training satellite images used for model development.
- *test_images.zip*: contains the test satellite images.
- *sample_submission.csv*: provides an example of the required output format for segmentation predictions. This file was not used in the current project.

Not all annotation records contain a segmentation mask. Approximately **53.35% of the annotation records contain valid RLE masks**, while the remaining records have missing *EncodedPixels* values. These missing values indicate that no segmentation mask is provided for that particular cloud category in the corresponding image.

The satellite images were obtained from **NASA Worldview** and were captured by the **TERRA and AQUA** polar-orbiting satellites using the **MODIS** instrument. Because an image may be constructed from two satellite orbits, an uncovered area between successive orbits can appear as a **black band**. This area is not part of the valid image region and must not be considered as a cloud prediction area.

The ground-truth annotations were created through a crowdsourcing process involving **68 scientists**. Each image was labeled by approximately three scientists, and the final ground truth was obtained from the union of their annotations after removing the black band areas.

The dataset therefore provides satellite imagery and pixel-level cloud annotations that can be used to formulate the project as a **multi-label image segmentation task**.

1.1.2. Dataset Structure

The training annotations are stored in a CSV file containing two main columns: *Image_Label* and *EncodedPixels*.

	Image_Label	EncodedPixels
0	0011165.jpg_Fish	20056 257 20440 257 20824 257 21208 257 21592 ...
1	0011165.jpg_Flower	102135 1 102245 275 102629 275 103013 275 1033...
2	0011165.jpg_Gravel	NaN
3	0011165.jpg_Sugar	NaN
4	002be4f.jpg_Fish	17668 241 18051 242 18435 242 18819 242 19203 ...

Figure 1: Initial dataframe of the training dataset

The *Image_Label* column contains both the image identifier and the corresponding cloud category. To make the data easier to process, this column was split into two separate columns: *Image_id* and *Cloud_Type*.

	Image_id	Cloud_Type	EncodedPixels
0	0011165.jpg	Fish	20056 257 20440 257 20824 257 21208 257 21592 ...
1	0011165.jpg	Flower	102135 1 102245 275 102629 275 103013 275 1033...
2	0011165.jpg	Gravel	NaN
3	0011165.jpg	Sugar	NaN
4	002be4f.jpg	Fish	17668 241 18051 242 18435 242 18819 242 19203 ...

Figure 2: Dataset structure after separating image identifiers and cloud categories

For example, an entry such as *0011165.jpg_Fish* is separated into the image identifier *0011165.jpg* and the cloud category *Fish*.

Each image can appear up to four times in the annotation file, once for each cloud category: Fish, Flower, Gravel, and Sugar.

The *EncodedPixels* column contains the segmentation mask information using Run-Length Encoding (RLE). When a cloud of a given category is present and annotated, its pixel locations are stored as an RLE sequence. Missing values in this column indicate that no mask is available for that particular image-category pair.

The RLE-encoded annotations were later decoded and converted into multi-channel segmentation masks for model training. The detailed preprocessing steps are described in Chapter 2.

This structure allows each satellite image to be associated with a corresponding pixel-level target containing the segmentation information for all four cloud categories.

1.1.3. Class Distribution

The distribution of available segmentation masks was analyzed for the four cloud categories: Fish, Flower, Gravel, and Sugar. The analysis shows that the number of annotated masks is not exactly equal across the four classes.

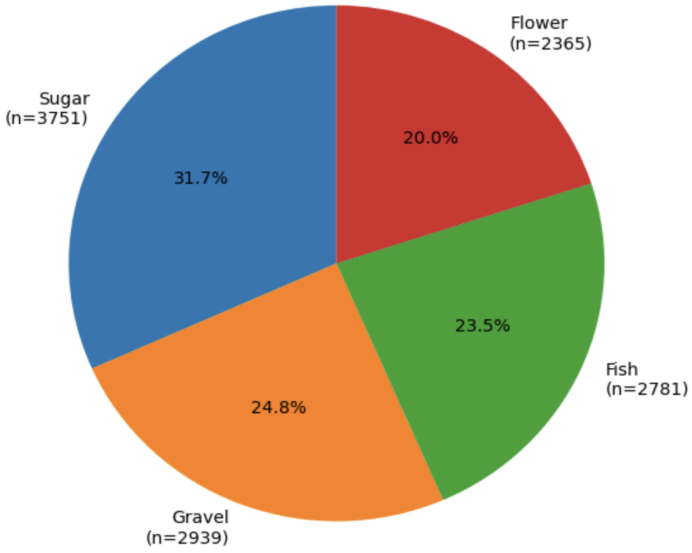


Figure 1: Distribution of available segmentation masks by cloud type

The pie chart shows the proportion of available masks belonging to each cloud category. Sugar represents the largest proportion, while Flower has the smallest proportion.

Overall, the dataset shows a moderate class imbalance, with Sugar having the highest number of available masks and Flower the lowest. Because the imbalance is not severe, no additional class-balancing technique was applied during preprocessing or model training. In cases of stronger class imbalance, techniques such as class-weighted loss could be used to give greater importance to underrepresented classes. However, for this dataset, the

observed imbalance was considered moderate enough to allow the model to learn from the original class distribution without introducing additional weighting.

1.1.4. Mask Availability and Cloud Categories per Image

The training dataset contains 22,184 image-category annotation records. Among these records, 11,836 contain segmentation masks, while 10,348 have missing EncodedPixels values. Therefore, 53.35% of the annotation records contain an available segmentation mask, while 46.65% do not. A missing value indicates that no annotated segmentation mask is available for that particular cloud category in the corresponding image. However, this does not mean that the image contains no clouds, since each image can contain one or more of the four cloud categories. The number of cloud categories represented by available annotations was also analyzed for each image.

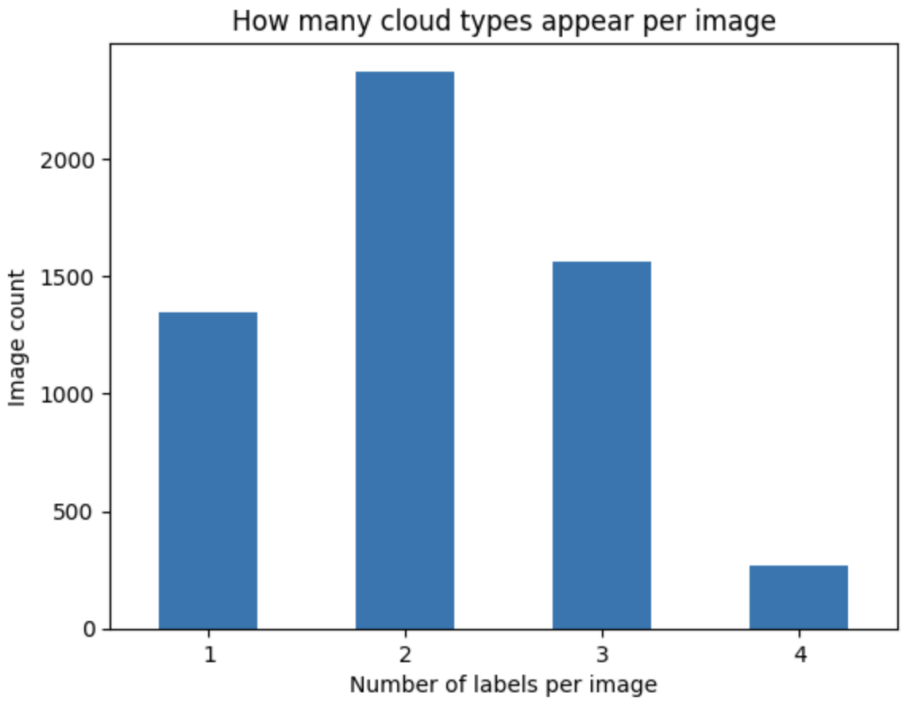


Figure 2: Number of Cloud Types per Image

The count plot shows the number of cloud types represented by available annotations in each image. The results show that images contain between one and four cloud types. Most images contain two cloud types, followed by three and one cloud type, while only a small number of images contain all four cloud types.

This distribution shows that most satellite images contain multiple cloud types, highlighting the complexity of the segmentation task. The model therefore needs to identify and distinguish several cloud categories within the same image.

1.1.5. Sensor Gap Analysis

During the exploration of the satellite images, a black band was observed in some images. This band is caused by an uncovered area between successive satellite orbits and represents a sensor gap rather than a valid image region.

The sensor gap was analyzed to understand the structure of the satellite images and to distinguish the black band from actual cloud regions.

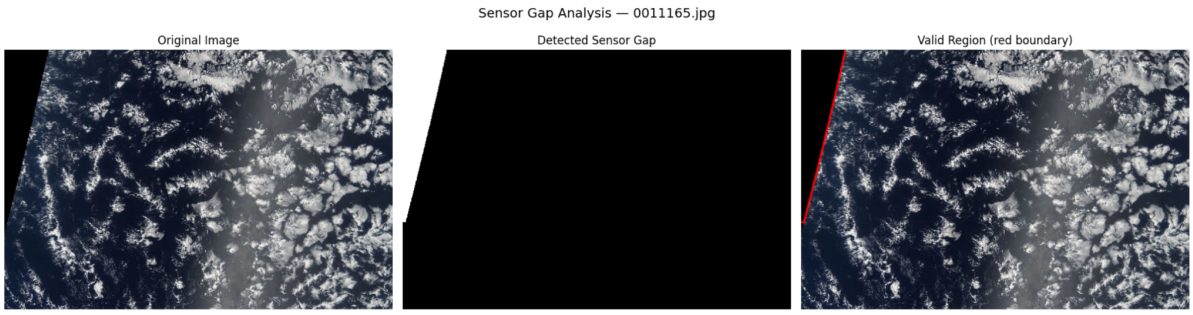


Figure 3: Example of sensor gap detection

What the figure shows:

- Left: Original satellite image.
- Middle: Detected sensor gap (white = gap).
- Right: Valid image region with its boundary.

The visualization shows that the black band does not contain valid satellite information and should not be interpreted as a cloud region. The ground truth annotations already exclude these areas.

In the modeling stage, no additional sensor-gap mask was applied during training or prediction. This approach avoids introducing errors from heuristic pixel-based gap detection and keeps the modeling process focused on the provided ground truth annotations.

1.2. Data Preparation

1.2.1. RLE Decoding

The segmentation masks in the training dataset are provided in Run-Length Encoding (RLE) format through the EncodedPixels column. RLE represents the locations of segmented pixels using pairs of **start positions** and **run lengths**.

To use these annotations for image segmentation, the RLE-encoded values were decoded into binary masks. Each decoded mask has the same spatial dimensions as the processed images, 384 × 576 pixels. Pixels belonging to the corresponding cloud category are assigned a value of 1, while the remaining pixels are assigned a value of 0.

Missing EncodedPixels values were treated as empty masks, since they indicate that no segmentation mask is available for that image-category pair.

The decoded binary masks were then used as the basis for creating the multi-channel ground truth masks described in the next section.

1.2.2. Multi-channel Mask Creation

After decoding the RLE annotations, the binary masks were combined into a single multi-channel mask for each satellite image. Four channels were created, one for each cloud category: Fish, Flower, Gravel, and Sugar.

Each channel represents the segmentation mask of one cloud category, with 1 indicating the presence of the corresponding cloud type and 0 indicating background pixels. Therefore, each processed image is associated with a four-channel ground truth mask with a shape of 384 × 576 × 4.

This representation allows the model to predict the four cloud categories independently for each pixel and provides the ground truth required for multi-label cloud segmentation.

1.2.3. Image and Ground Truth Mask Visualization

To verify the segmentation annotations, a selected satellite image was visualized together with its corresponding ground truth masks. The masks were displayed separately for each cloud category to examine their spatial alignment with the original image.

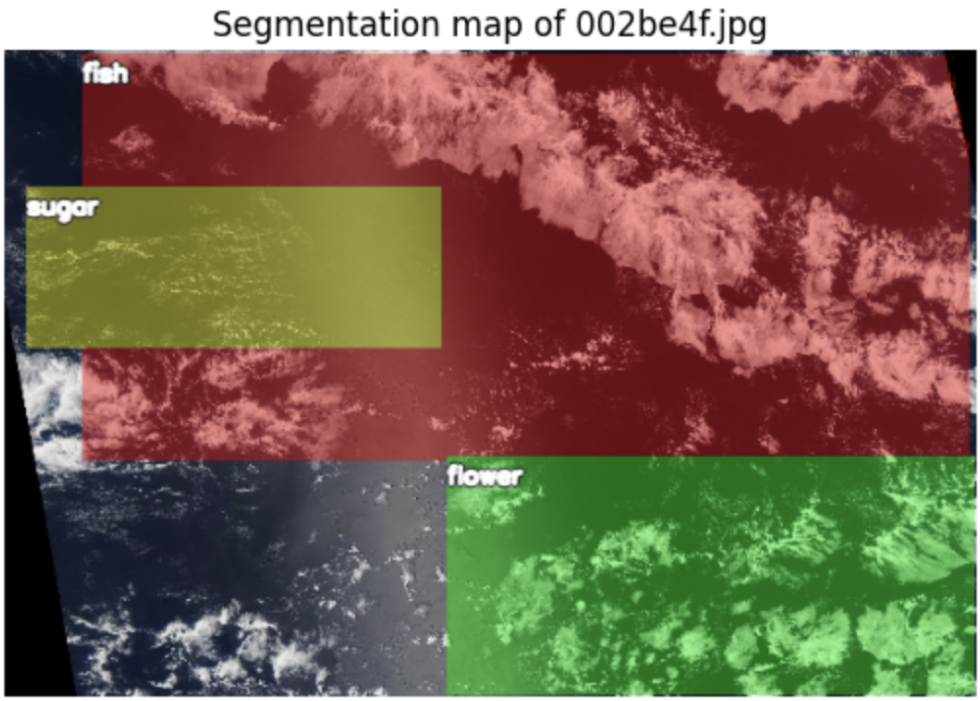


Figure 4: Satellite image 002be4f.jpg with ground truth segmentation masks

The visualization shows the satellite image 002be4f.jpg together with its available ground truth segmentation masks. The masks illustrate the spatial distribution of the annotated cloud regions and their alignment with the original image.

The black band visible in the lower-left corner represents the sensor gap. It is not part of the valid image region, and the ground truth annotations already exclude this area.

This visualization provides a visual check of the segmentation annotations and confirms their alignment with the satellite image before model training.

1.2.4. Image Resizing

The original satellite images have a resolution of 1400 × 2100 pixels. To reduce the computational cost and make the images suitable for model training, the images were resized to 576 × 384 pixels.

The same spatial dimensions were used for the corresponding ground truth masks to ensure that the images and masks remained correctly aligned. After resizing, each image had a shape of 384 × 576 × 3, while each ground truth mask had a shape of 384 × 576 × 4.

This resizing step reduced the input size while preserving the spatial information required for cloud segmentation and made the dataset more suitable for efficient model training.

1.2.5. Normalization

After resizing, the satellite images were normalized before being used as input to the neural networks. ImageNet normalization was applied using the mean values (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225) for the RGB channels.

This normalization was used because the models use ImageNet-pretrained encoders. It provides input values with a distribution compatible with the pretrained weights and supports more stable model training.

The ground truth masks were not normalized. They were kept as binary values, where 1 represents the presence of a cloud category and 0 represents its absence.

1.2.6. Data Augmentation

Data augmentation was applied only to the training data to increase the diversity of the input images and improve the model's ability to generalize to unseen images. The validation and test data were not augmented; they were only resized and normalized.

The main purpose of data augmentation is to expose the model to realistic variations of the training images. As described by Géron (2025), transformations such as shifting, rotating, resizing, flipping, and modifying brightness or contrast can increase the variety of training examples and act as a regularization technique. In our project, augmentation was also intended to make the model more tolerant to variations in the position, orientation, and visual appearance of cloud formations.

In contrast to physically adding augmented images to the dataset, we used on-the-fly augmentation. The transformations are applied inside the dataset whenever a training image is requested. Therefore, the number of training images remains unchanged, while the model can see a different transformed version of an image during different epochs. This provides additional variation without increasing the size of the dataset stored on disk.

Albumentations Library

The Albumentations library was selected to implement the augmentation pipeline. It is particularly suitable for image segmentation because it can apply geometric transformations consistently to both an image and its corresponding masks. This is important because the spatial alignment between the input image and the segmentation labels must be preserved.

Geometric transformations such as flipping and rotation are therefore applied identically to the image and all corresponding masks. Photometric transformations, such as brightness and contrast adjustments, are applied only to the image because they change its visual appearance but should not modify the segmentation labels.

At an early stage of the project, the imgaug library was also considered. Albumentations was selected because it provided the required augmentation functionality and was suitable for the Python environment used in the project.

For reproducibility, it is important to distinguish between the package installed in the project environment and the import name used in the code. The project installs albumentationsx, as specified in requirements.txt. The library provides a compatible Albumentations interface and is imported in the code as albumentations. Therefore, the implementation uses:

```
import albumentations as A
```

Augmentation Pipeline

Each transformation has its own probability p. Consequently, the training images can receive different combinations of transformations during different calls to the dataset.

The final augmentation pipeline contains four transformations:

- HorizontalFlip (p = 0.3)
- VerticalFlip (p = 0.3)
- RandomBrightnessContrast (p = 0.8)
- Rotate (p = 0.6)

Horizontal and vertical flipping are appropriate for satellite images because these images do not have a fixed canonical orientation. A flipped satellite image remains a plausible representation of the observed scene.

Rotation was used for a similar reason. Cloud formations can occur in different orientations, and rotation helps reduce the model's dependence on a particular spatial orientation.

RandomBrightnessContrast was used to introduce moderate variations in image brightness and contrast. This can simulate differences in illumination and imaging conditions while preserving the relevant visual characteristics of the cloud classes. A relatively high probability of 0.8 was chosen, while the default moderate transformation limits were retained to avoid excessively changing the visual information in the images.

The transformation probabilities were selected heuristically as a reasonable initial configuration. They were not systematically optimized and could therefore be investigated further in a future optimization step.

The augmentation pipeline was implemented as follows:

```
import albumentations as A

pipeline = A.Compose([
   A.HorizontalFlip(p=0.3),
   A.VerticalFlip(p=0.3),
   A.RandomBrightnessContrast(p=0.8),
   A.Rotate(p=0.6)
])

# Call the pipeline giving it an image and the stack of its masks
# NOTE: `csi` is an instance of our custom class `CloudSegmentationImage` which
# gathers the image together with the labels (the masks)
transformed_data = pipeline(
   image=csi.get_image_rgb(),
   masks=csi.get_mask_stack()
)
```

Figure 7: Python code used to define the Albumentations augmentation pipeline.

Handling of Segmentation Masks

Special care was taken to ensure that the segmentation masks remained correctly aligned with the transformed images. When a geometric transformation is applied, the same transformation is applied simultaneously to the image and its corresponding masks.

For rotation, the newly created corner regions are filled with the background value. In the masks, this corresponds to a value of 0, so no artificial cloud regions are introduced during the transformation.

The masks are also resampled using nearest-neighbour interpolation. This is important for segmentation because the mask values represent class labels and should not be replaced by interpolated intermediate values.

Application to the Training and Validation Datasets

The augmentation pipeline is passed to the custom CloudSegmentationDataset and is applied in the **getitem** method whenever a training sample is requested. This means that augmentation is performed on the fly during training.

```
train_dataset = CloudSegmentationDataset(
    train_labels_df, TRAIN_PATH, pipeline=pipeline
)

# No augmentation for validation, so it reflects real-world performance
val_dataset = CloudSegmentationDataset(val_labels_df, TRAIN_PATH, pipeline=None)
```

Figure 8: Application of the augmentation pipeline to the training dataset and its exclusion from the validation dataset.

Augmentation was deliberately excluded from the validation and test sets. This ensures that their evaluation represents the model's performance on unmodified images and that the validation score remains comparable across epochs. Otherwise, changes in the validation images caused by random augmentation could affect the measured score independently of changes in model performance.

1.3. Data Preparation Results

1.3.1. Augmented Images

The figure below shows one image with its mask overlay, once in the original and once after passing through the pipeline. Note how the masks follow the geometric transformations of the image:

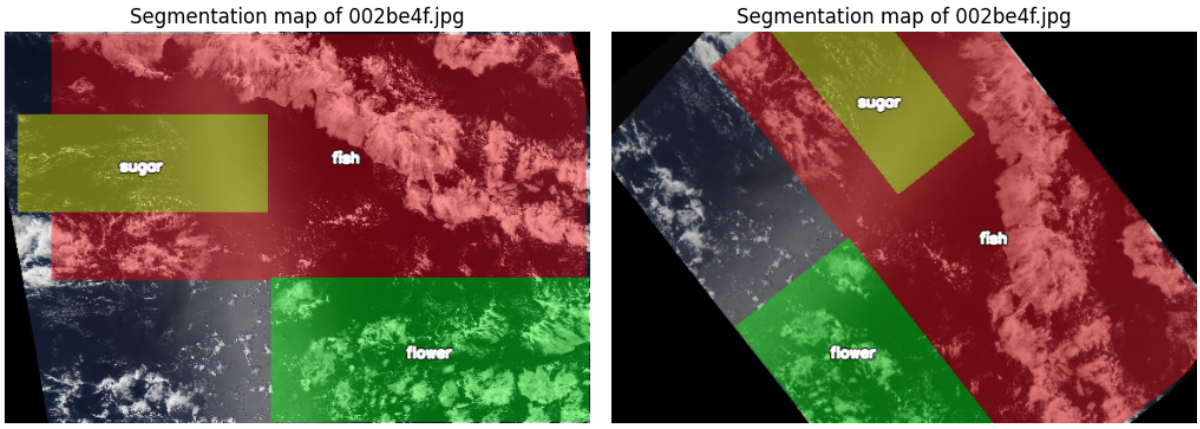


Figure 5: Original image with masks compared to an augmented version

How much the augmentation actually contributes to the model quality is discussed together with the training results in the model chapters.

1.3.2. Train/Validation Split

The dataset was divided into two subsets: **90% for training and 10% for validation**. The split was performed at the image level using a fixed random state of 42.

- **Training images:** 4,991
- **Validation images:** 555
- **Total:** 5,546 images

The validation set was used to evaluate the model during training and to monitor its performance on unseen images.

1.3.3. Data Loaders

The training and validation datasets were loaded using PyTorch *DataLoader*. A batch size of **8** was used for both datasets.

The training DataLoader shuffled the training samples to provide a different order of images during each epoch. The validation DataLoader did not shuffle the data because the order of validation samples does not need to be randomized.

The DataLoaders were configured with num_workers=0 and *pin_memory=True* to control data loading and support efficient data transfer to the GPU.

1.3.4. Batch Size

A batch size of **8** was used for both the training and validation datasets. This means that the model processes **8 images at a time** before updating its weights during training.

Using a batch size of 8 provides a balance between computational efficiency and GPU memory usage.

1.3.5. Data Format

After preprocessing, the images have the shape (3, 384, 576) and the corresponding masks have the shape (4, 384, 576). The images contain three RGB channels, while the masks contain four channels corresponding to Fish, Flower, Gravel, and Sugar.

The data is then loaded by the DataLoader in batches of 8. Therefore, the final input shapes used by the model are:

- Images: [8, 3, 384, 576]
- Masks: [8, 4, 384, 576]

Here, 8 represents the batch size, 3 represents the RGB channels, and 4 represents the four cloud classes.

3. Cloud Segmentation Models 1: Cloud Segmentation Using U-Net

3.1. Executive Summary

This section describes the U-Net method used in the cloud-segmentation project. It explains why it is suitable for the task, documents the network architecture applied in the project, and interprets the available validation results.

The current model performs four-channel segmentation, producing independent Fish, Flower, Gravel, and Sugar masks for each 384 × 576 RGB image.

The learning curves show consistent improvement over 15 epochs. Training and validation losses both decrease and remain close to one another, while Dice scores increase. This indicates that the model is learning and there is no strong sign of overfitting in the displayed run.

However, the final validation Dice coefficient of 0.2462 is still modest, so the current model should be treated as a baseline that requires further improvement before strong segmentation accuracy can be claimed.

The trained model was also applied to four unseen Kaggle test images and produced separate Fish, Flower, Gravel, and Sugar masks; these predictions provide a qualitative demonstration only because ground-truth test masks are not available.

Key validation results:

- Dice = 0.2462
- IoU = 0.3466
- loss = 1.0770

These metrics were computed with different definitions in the project code, so Dice and IoU should not be compared as if they were mathematically paired values

3.2. Theoretical Background of the U-Net Model

3.2.1. Description of U-Net Method

U-Net is a convolutional neural network architecture designed for image segmentation. Instead of assigning one label to an entire image, it produces a spatial prediction at pixel level. Its characteristic U-shaped structure combines a contracting encoder with an expanding decoder and connects corresponding levels with skip connections.

Component	Role in U-Net	Role in this project
Encoder	Extracts increasingly abstract image features while reducing spatial resolution.	Learns cloud texture, shape, brightness, and larger spatial patterns.
Bottleneck	Represents the most compressed and high-level feature description.	Combines broad contextual information about cloud organization.
Decoder	Upsamples the compressed representation back toward the original image size.	Reconstructs the location and extent of cloud regions.
Skip connections	Pass fine spatial detail directly from encoder layers to matching decoder	Help preserve boundaries and local cloud structure that could be lost during
Output layer	Produces one or more segmentation maps.	Produces four sigmoid probability maps: Fish, Flower, Gravel, and Sugar.

Figure 6: Core U-Net components and their role in the cloud project comparison

For the current four-mask task, the output is multi-label rather than softmax multiclass segmentation. A pixel can receive independent probabilities for the four cloud masks, so the final layer uses four sigmoid channels. This differs from the earlier binary version of the project, where the four masks were merged into one cloud/non-cloud mask.

3.2.2. Common Applications of U-Net

U-Net was originally introduced for biomedical image segmentation, but its encoder-decoder structure is useful in many domains where object location and shape matter. Typical applications include:

- Medical imaging: segmenting organs, tumors, lesions, and cells.
- Remote sensing and Earth observation: mapping clouds, water, vegetation, land cover, roads, and buildings.
- Agriculture and environmental monitoring: crop boundaries, disease areas, burned regions, and habitat mapping.
- Autonomous and robotic vision: separating roads, vehicles, pedestrians, and other scene components.
- Industrial inspection: locating defects, cracks, or damaged regions in imagery.

3.2.3. Rationale for using the U-Net for this Cloud Project

The project requires spatial masks, not only image-level labels. It must determine where cloud-pattern regions occur in each satellite image. U-Net is therefore appropriate because it returns a prediction for each pixel while preserving spatial structure.

Project requirement	Why U-Net fits
Pixel-level output	U-Net produces full-resolution segmentation maps rather than a single image label.
Irregular cloud regions	Convolutions learn local texture while deeper layers capture larger cloud structures.
Need for spatial detail	Skip connections restore fine information lost during downsampling.
Four cloud masks	The final layer can be changed from 1 output channel to 4 independent channels.
Moderate training-data size	Data augmentation and a compact U-Net can be trained without requiring a very large foundation model.

Figure 7: Project requirements and reasons for using U-Net

3.3. Application of the U-Net Model

3.3.1. Application of U-Net in this Project

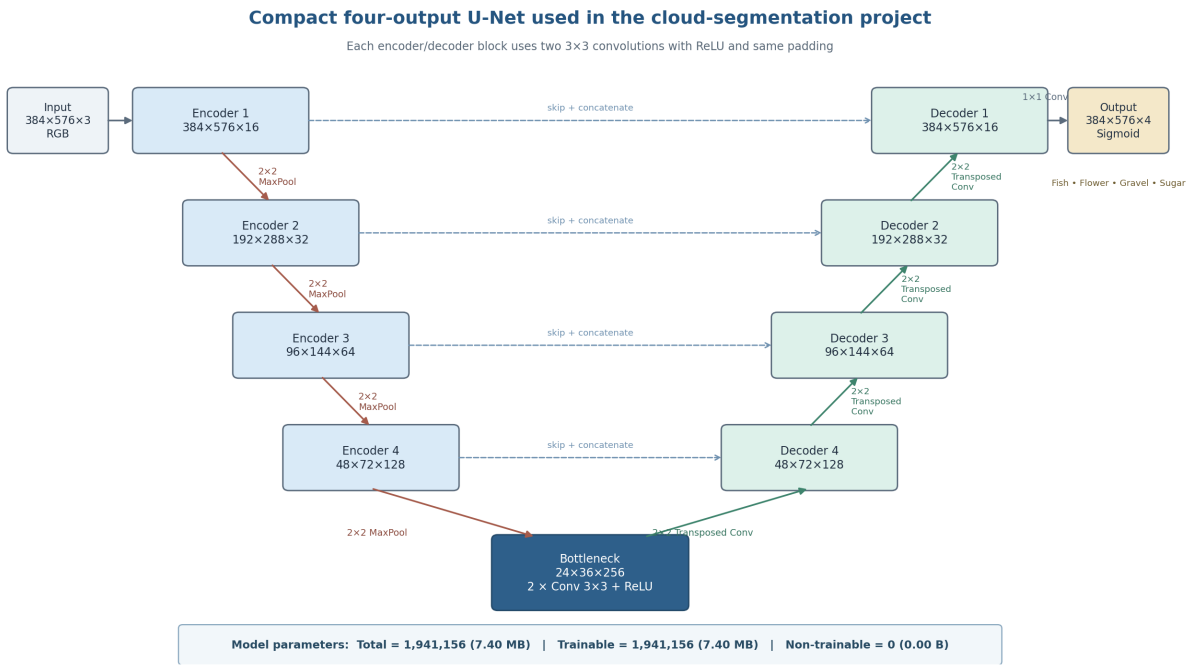


Figure 8: U-Net architecture used in the four-class cloud-segmentation model

U-net Architecture. The project uses a compact U-Net that receives a 384 × 576 RGB satellite image and returns four full-resolution probability masks, one for each cloud class: Fish, Flower, Gravel, and Sugar. The architecture preserves the standard U-Net encoder–bottleneck–decoder structure with skip connections, while using a relatively small number of filters to keep the model computationally manageable.

Input and encoder. The input is a 384 × 576 × 3 RGB image. Four encoder levels progressively learn richer cloud features. Each level applies two 3 × 3 convolutions with ReLU activation and same padding, followed by 2 × 2 max pooling. The number of feature

channels increases from 16 to 32, 64, and 128, while the spatial resolution is reduced from 384 × 576 to 48 × 72.

Bottleneck. After the final pooling operation, the bottleneck operates at 24 × 36 pixels with 256 feature channels. It contains two 3 × 3 convolutional layers and captures the most compressed, high-level representation of the cloud patterns.

Decoder and skip connections. The decoder restores spatial resolution using 2 × 2 transposed convolutions. At each level, the upsampled decoder features are concatenated with the corresponding encoder features through a skip connection. These skip connections help recover spatial detail and boundaries that may be weakened during pooling. The decoder channels decrease from 128 to 64, 32, and 16 as the image returns to its original resolution.

Output layer. A final 1 × 1 convolution produces four channels with sigmoid activation. Each channel is an independent probability map for Fish, Flower, Gravel, or Sugar. Sigmoid is appropriate because the cloud masks are treated independently rather than forcing every pixel to belong to exactly one class.

Model size. The network contains 1,941,156 parameters (7.40 MB). All 1,941,156 parameters are trainable, and there are 0 non-trainable parameters. This matches the architecture shown in Scheme 1 and confirms that the model does not include layers with stored non-trainable statistics such as Batch Normalization in this baseline configuration.

3.3.2. Results and Interpretation

The loss curves decrease consistently. Training loss falls from approximately 1.29 at the beginning to about 1.08 by the final displayed epoch, while validation loss falls from approximately 1.26 to 1.077. The two curves remain close throughout training. This is a positive sign because it indicates that the model is improving on both training and unseen validation data, with no clear divergence that would indicate strong overfitting.

The Dice coefficient also improves. Training Dice rises from about 0.14 to roughly 0.24, and validation Dice rises to approximately 0.246. The validation curve fluctuates slightly, which is normal, but the overall trend remains upward. The final validation Dice being slightly above training Dice is not concerning in this setup; training data are augmented on the fly, which can make training batches harder than the unaugmented validation images.

The important limitation is the absolute Dice level. A Dice value around 0.25 indicates that predicted cloud-pattern regions still overlap the ground truth only modestly. Therefore, the learning trend is encouraging, but the segmentation quality is not yet strong.

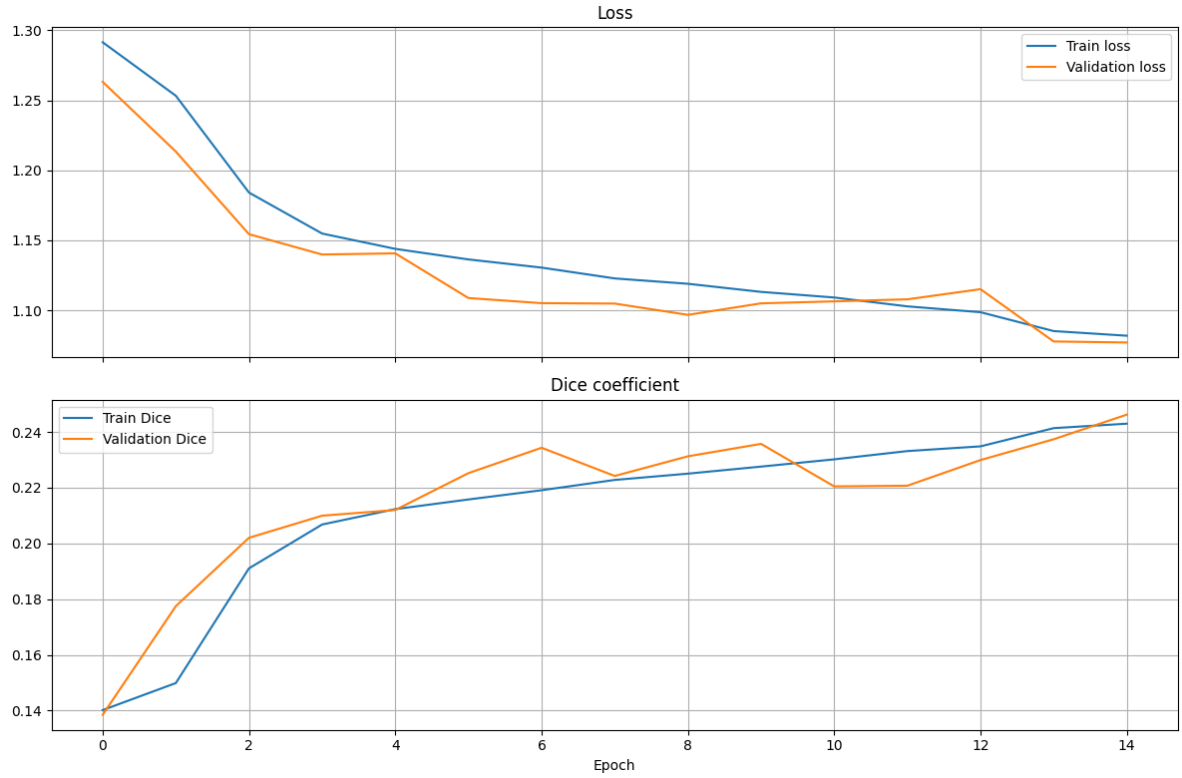


Figure 9: Training and validation loss and Dice coefficient across 15 displayed epochs.

3.3.3. Final validation metrics

Metric	Validation value	Interpretation
Dice coefficient	0.2462	Higher is better. The current overlap is modest and indicates room for substantial improvement.
IoU coefficient	0.3466	Higher is better. This value is threshold-based in the current implementation and should be interpreted independently of the soft Dice value.
Loss	1.0770	Lower is better. The reduced loss relative to early epochs confirms that optimization progressed.

Figure 10: Model 1 Final validation metrics.

Taken together, the results show successful training behavior but limited final segmentation accuracy. The absence of a large train–validation gap suggests that the main issue is not obvious overfitting. Instead, the model may still be underfitting the complexity of the four cloud patterns, may be affected by class imbalance, or may require better thresholding and stronger feature extraction.

Observation	What it means
Loss decreases for train and validation	The optimizer is learning useful parameters and generalization is improving during the displayed epochs.
Train and validation curves remain close	No strong evidence of overfitting is visible in this run.
Dice increases but remains low	The model is learning the task, but predicted masks still have limited overlap with reference masks.
Validation Dice slightly exceeds train Dice	This can occur because training images are augmented while validation images are not.
IoU is numerically above Dice	The project uses different aggregation/threshold rules for the two metrics, so the two values should not be directly compared.

Figure 11: Model 1 Interpretation of training and validation behavior.

3.3.4. Predictions on unseen test images

To examine how the trained four-channel U-Net behaves on data that were not used for training or validation, the model was applied to four randomly selected Kaggle test images. For each image, the model produced four binary segmentation masks, corresponding to Fish, Flower, Gravel, and Sugar. White areas indicate pixels predicted as belonging to the named cloud-pattern class, while black areas indicate pixels not assigned to that class.

The predictions differ substantially among the four images, showing that the model does not return the same class pattern for every scene. In the first image (90897a5.jpg), Fish and Flower are almost absent, while large regions are assigned to Gravel and Sugar. In the second image (d6af239.jpg), the model predicts clear Fish regions and smaller Flower, Gravel, and Sugar regions. The third image (700e9bc.jpg) contains predicted Fish and Flower regions, almost no Gravel, and only limited Sugar. In the fourth image (0078e5a.jpg), Fish is the most prominent predicted class, while Flower is nearly absent and Gravel and Sugar occur only in localized areas.

These examples confirm that the model can generate four independent spatial masks on unseen images and can assign different cloud-pattern combinations to different scenes. The results should not, however, be interpreted as a measurement of test accuracy. The Kaggle test images do not include ground-truth masks, so Dice, IoU, false-positive rates, and false-negative rates cannot be computed for these examples. Quantitative assessment therefore remains based on the labeled validation set, while Figure 2 serves as a qualitative check of the model's behavior on unseen data.

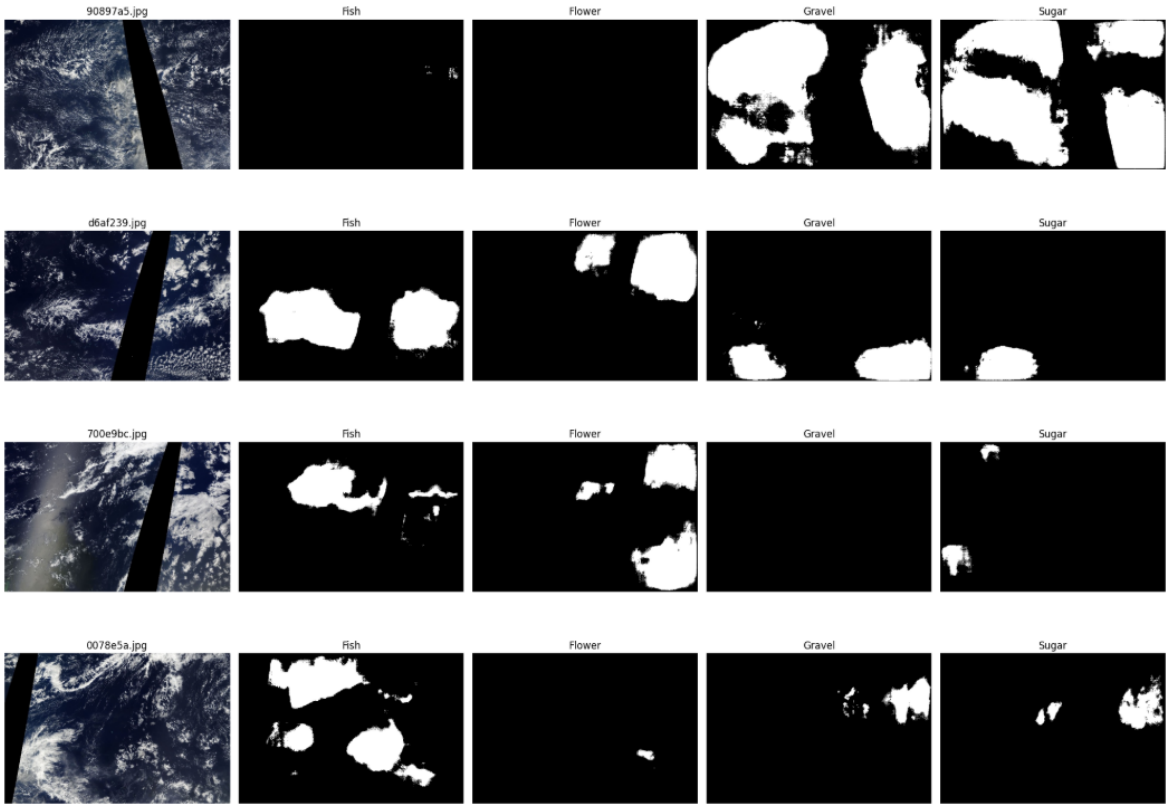


Figure 12: Model 1 predicted cloud pattern masks for randomly selected test images

3.4. Conclusion

U-Net is well suited to this project because the task requires spatial cloud masks rather than only image-level labels. The encoder-decoder structure and skip connections allow the model to combine broad cloud context with fine spatial detail. The implementation was adapted from binary segmentation to four independent cloud-pattern channels by preserving Fish, Flower, Gravel, and Sugar masks separately and using a four-channel sigmoid output layer.

The current validation run shows stable learning: both losses decrease and Dice improves without a clear sign of overfitting. Nevertheless, the final Dice coefficient of 0.2462 indicates that segmentation overlap remains limited. The current model should therefore be considered a baseline. The qualitative predictions on four unseen test images demonstrate that the model can produce distinct Fish, Flower, Gravel, and Sugar masks for new scenes, but these images cannot provide quantitative test accuracy because their ground-truth masks are unavailable. Per-class evaluation, threshold optimization, qualitative validation-mask inspection, and comparison with stronger U-Net variants remain the most important next steps.

3.4.1. Results Still Needed for a Complete Final Model Assessment

The available results now include the training curves, overall validation metrics, and qualitative predictions on four unseen test images. Several additional quantitative results would still strengthen a final scientific or project report:

- Per-class Dice and IoU for Fish, Flower, Gravel, and Sugar. These results show whether one cloud class is much harder than the others.
- A threshold-selection result on the validation set. A global threshold can be used for simple comparison, followed by per-class threshold optimization for the selected final model.
- Qualitative validation examples showing: original image, true mask, predicted probability, and predicted mask for each class.
- A model-selection table if additional architectures are trained. Use mean validation Dice as the primary selection metric, mean validation IoU as secondary, and validation loss as supporting information.

3.4.2. Recommended Next Improvements

Priority	Action	Reason
1	Calculate per-class Dice/IoU and inspect class frequencies.	Identifies weak classes and possible imbalance.
2	Optimize the segmentation threshold on validation data.	The default 0.5 cutoff may not be optimal; class-specific thresholds may improve final
3	Compare a stronger U-Net variant (e.g., more filters, BatchNorm, or pretrained encoder).	Current Dice suggests the baseline may not capture enough discriminative features.
4	Review augmentation strength and class balance.	Ensures training data remain realistic and minority cloud classes are represented.
5	Save model, CSV training history, split lists, thresholds, and configuration.	Makes results reproducible after Colab/kernel restarts.

Figure 13: Model 1 Recommended next improvements.

4. Cloud Segmentation Models 2: Cloud Segmentation Using U-Net with ResNet50

4.1. Executive Summary

The second deep learning model used in this project was a U-Net architecture with a ResNet50 encoder. The model was implemented using the *segmentation_models_pytorch* library with *ImageNet-pretrained* encoder weights.

Item	Value
Task	Semantic segmentation of cloud patterns in satellite images
Model	U-Net with a ResNet-50 encoder pre-trained on ImageNet (`segmentation_models_pytorch`)
Input	384 × 576 pixels, RGB
Output	Four independent masks: Fish, Flower, Gravel, Sugar (multi-label)
Data	5,546 labelled images, split 4,991 train / 555 validation
Framework	PyTorch, Albumentations, OpenCV

Figure 14: Model 2 Project overview

The same U-Net theory introduced in Chapter 3 applies to this model. To avoid repeating the theoretical background, this chapter focuses on the specific implementation, training configuration, results, and evaluation of the U-Net with ResNet50 used in this project.

4.2. Theoretical Background

The theoretical background of the U-Net architecture was introduced in Chapter 3. Therefore, this section focuses on the ResNet50 encoder used in this model.

ResNet50 is a convolutional neural network based on residual learning. It uses residual blocks with shortcut connections, which allow information to pass through the network more effectively and help train deeper networks.

In this model, ResNet50 is used as the encoder of the U-Net architecture and is initialized with ImageNet-pretrained weights. The encoder extracts hierarchical features from the input satellite images at different spatial resolutions. These features are then transferred to the U-Net decoder through skip connections, allowing the model to combine high-level semantic information with spatial details for pixel-level cloud segmentation.

The main components of the U-Net with ResNet50 used in this project and their roles are summarized in Figure 15.

Component	Role in U-Net + ResNet50	Role in this project
Encoder (ResNet50 backbone)	A deep CNN that progressively extracts features while reducing the spatial resolution of the input. ResNet50 uses residual blocks with shortcut connections to facilitate the training of a deep network.	Extracts hierarchical features from the satellite images, from low-level edges and textures to higher-level cloud patterns. ImageNet-pretrained weights provide a useful starting point for feature extraction.
Skip connections	Transfer intermediate feature maps from the encoder directly to corresponding decoder stages, helping preserve spatial information lost during downsampling.	Help the decoder recover spatial details and cloud boundaries by combining encoder features with the upsampled decoder features.
U-Net decoder	Progressively upsamples the deep encoder features and combines them with encoder features through skip connections to reconstruct the segmentation at higher spatial resolutions.	Restores the feature maps from the deepest representation back to the original image resolution of 384 × 576 pixels for pixel-level cloud segmentation.
Segmentation head	Applies a final convolution to the decoder output to produce the required number of segmentation channels.	Produces four independent output channels corresponding to Fish, Flower, Gravel, and Sugar. The outputs are converted to probabilities using sigmoid during evaluation.
Output masks	Each output channel represents the predicted probability of one cloud pattern for every pixel.	Generates four independent probability maps for Fish, Flower, Gravel, and Sugar, with a threshold of 0.5 used to obtain the final binary masks.

Figure 15: Core U-Net + ResNet50 components and their role in this project

4.2. Application of this U-Net model

Architecture as Instantiated

The model was implemented using the segmentation_models_pytorch library as a U-Net with a ResNet50 encoder. The encoder was initialized with ImageNet-pretrained weights and receives RGB satellite images with three input channels.

The architecture consists of three main components: the ResNet50 encoder, the U-Net decoder, and the segmentation head. The encoder progressively extracts higher-level features while reducing the spatial resolution of the input image. The decoder then progressively restores the spatial resolution and combines encoder features through skip connections to preserve spatial information.

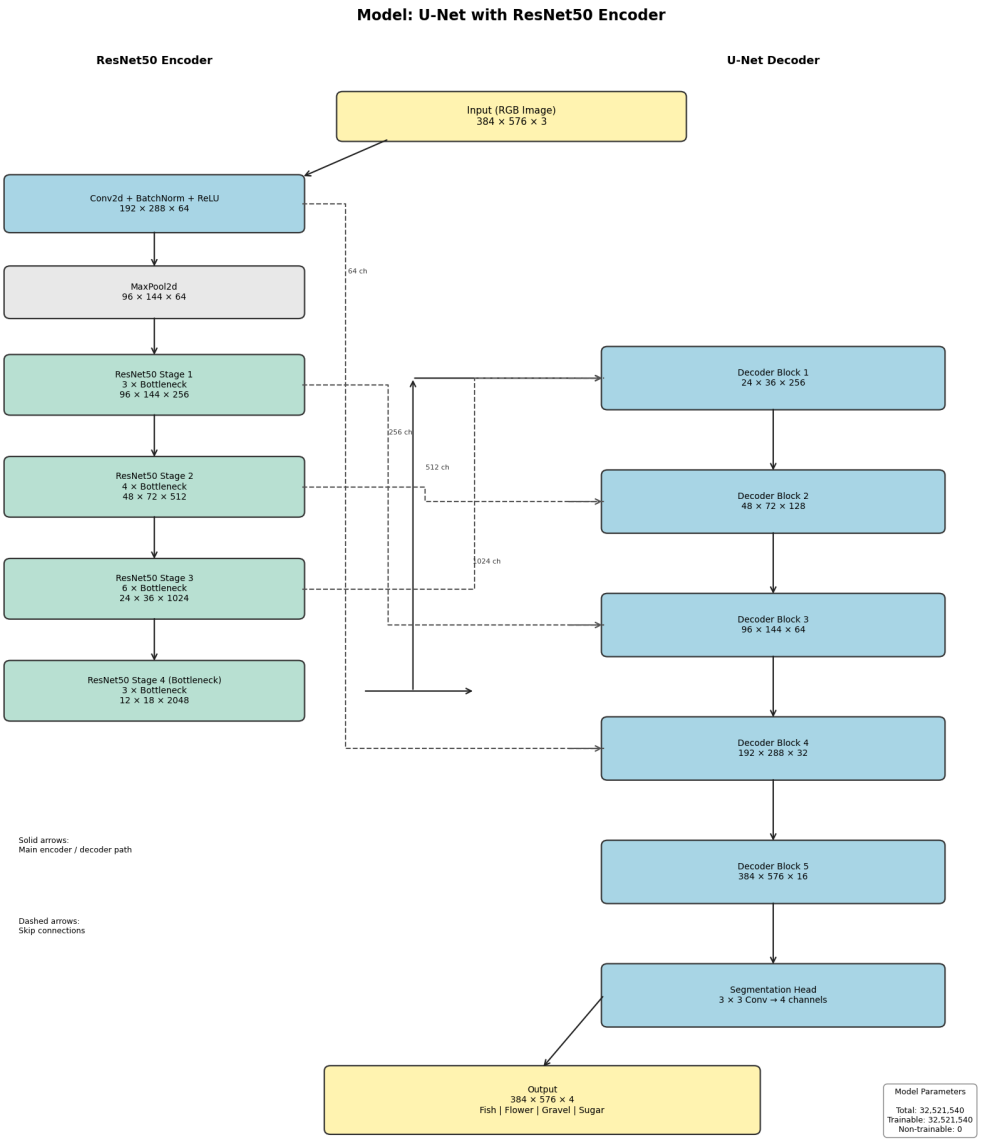


Figure 16: Model 2 instantiated U-Net with ResNet50 architecture

For an input image of 384 × 576 pixels, the encoder begins with a convolutional layer producing 64 channels at a spatial resolution of 192 × 288. After max pooling, the spatial resolution is reduced to 96 × 144. The ResNet50 encoder then progressively transforms the feature representation through stages with 256, 512, 1024, and 2048 channels, while reducing the spatial resolution to 48 × 72, 24 × 36, and finally 12 × 18 pixels. The deepest encoder representation therefore contains 2048 channels at a spatial resolution of 12 × 18.

The U-Net decoder progressively increases the spatial resolution from 12 × 18 to the original image resolution of 384 × 576. Encoder features are transferred to the corresponding decoder stages through skip connections and combined with the decoder representations.

These connections provide spatial information from earlier encoder stages and help the decoder reconstruct detailed cloud boundaries and regions.

Finally, the segmentation head applies a 3 × 3 convolution to produce four output channels corresponding to the four cloud categories: Fish, Flower, Gravel, and Sugar. Therefore, for a batch size of one, the final model output has the shape [1, 4, 384, 576].

The model contains 32,521,540 trainable parameters. All parameters are trainable, with 0 non-trainable parameters.

Figure 16 presents the detailed architecture of the U-Net with ResNet50, including the main encoder and decoder stages, feature-map dimensions, skip connections, and final segmentation output.

4.2.1. Loss Function

A custom **Dice-BCE loss function** was used to train the U-Net with ResNet50 model. It combines Binary Cross-Entropy (BCE) loss and Dice loss.

The total loss was calculated as a weighted combination of the two components, with a higher weight assigned to the Dice loss. The BCE loss had a weight of 0.3, while the Dice loss had a weight of 0.7.

The Dice component helps the model improve the overlap between the predicted cloud regions and the ground-truth masks, while the BCE component helps the model classify each pixel correctly.

The final loss was calculated as: Total Loss = 0.3 × BCE Loss + 0.7 × Dice Loss

4.2.2. Optimizer and Learning Rate

The **AdamW optimizer** was used to update the model parameters during training. The initial learning rate was set to $\mathbf{8 \times 10^{-5}}$, and the weight decay was set to $\mathbf{1 \times 10^{-4}}$.

AdamW was selected to provide stable parameter updates while weight decay helps reduce overfitting and improve generalization.

A **ReduceLROnPlateau** scheduler was also used. When the validation loss stopped improving for three consecutive epochs, the learning rate was reduced by a factor of 0.5.

4.2.3. Training Configuration

The model was trained for a maximum of **25 epochs** with a batch size of **8**. During training, the model was evaluated on the validation set after each epoch.

Early stopping was used with a patience of **5 epochs**. If the validation performance did not improve for five consecutive epochs, training was stopped to reduce unnecessary training and help prevent overfitting.

The best-performing model was saved during training based on its validation Dice score. The saved checkpoint was then used for the final validation evaluation and for generating predictions on the unseen test images.

4.2.4. Training Results

The model was trained for a maximum of 25 epochs. During training, both training loss and validation loss were monitored, together with the validation Dice score.
The training loss gradually decreased from 0.6636 in the first epoch to 0.5669 by epoch 20.

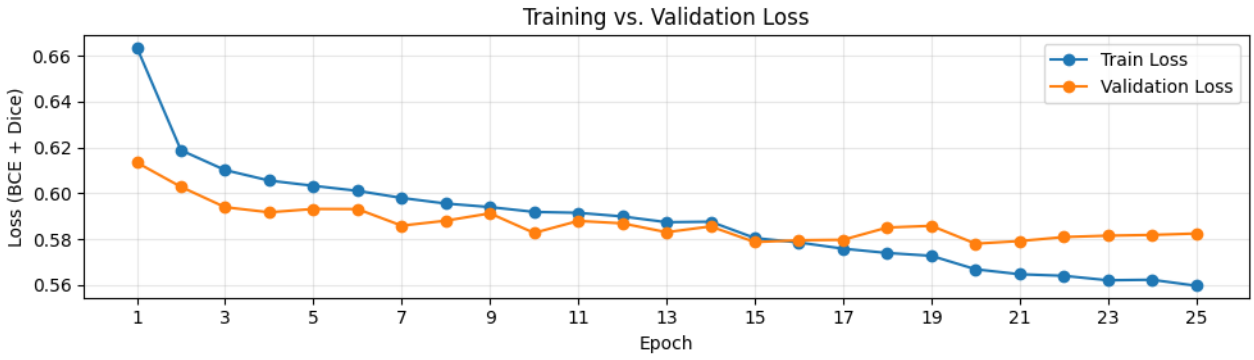


Figure 17: Model 2 Training and Validation Loss across 25 Epochs

The validation loss also improved during training, reaching its lowest recorded value of 0.5781 at epoch 20, while showing small fluctuations between epochs.

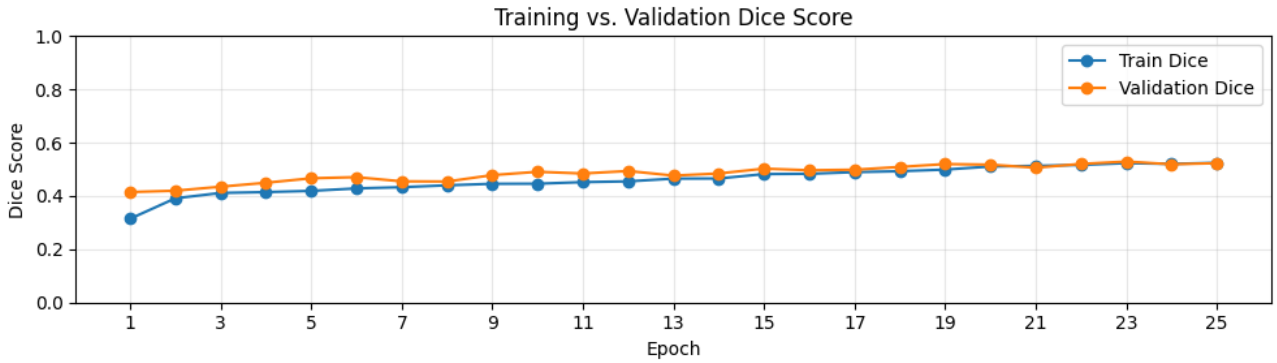


Figure 18: Model 2 Validation Dice Score across the training epochs

The validation Dice score generally improved during training, reaching its highest recorded value of 0.5198 at epoch 19. After this point, the validation Dice score slightly decreased to 0.5177 in epoch 20. The training results show an overall improvement in the model's learning performance, although the validation metrics fluctuated between epochs.

4.3. Model Evaluation

4.3.1. Final Validation Metrics

After training, the saved best-performing U-Net with ResNet50 checkpoint was reloaded and evaluated on the validation set. The final evaluation was based on the predicted binary masks obtained using a sigmoid activation and a threshold of 0.5.

The evaluation used the Dice score and Intersection over Union (IoU). The Macro Dice score averages the Dice scores of the four cloud categories, giving each category equal importance. The Mean IoU similarly summarizes the IoU values across the four categories.

The overall Macro Dice score was 0.6118, while the Mean IoU was 0.4432. These results indicate that the model achieved a moderate level of segmentation performance on the validation images. The per-class validation results are shown below.

Cloud Class	Dice Score	IoU
Fish	0.5478	0.3772
Flower	0.6962	0.5340
Gravel	0.5729	0.4014
Sugar	0.6301	0.4600
Macro / Mean	**0.6118**	**0.4432**

Figure 19: Model 2 Cloud Class, Dice Score and IoU comparison

The per-class results show differences in segmentation performance across the four cloud categories. Flower achieved the highest Dice score at 0.6962, while Fish achieved the lowest at 0.5478. Gravel and Sugar achieved Dice scores of 0.5729 and 0.6301, respectively.

These differences indicate that the model segmented the Flower class more effectively than the other cloud categories, whereas Fish remained more challenging. Overall, the model was able to detect all four cloud categories, although the quality of segmentation varied between classes.

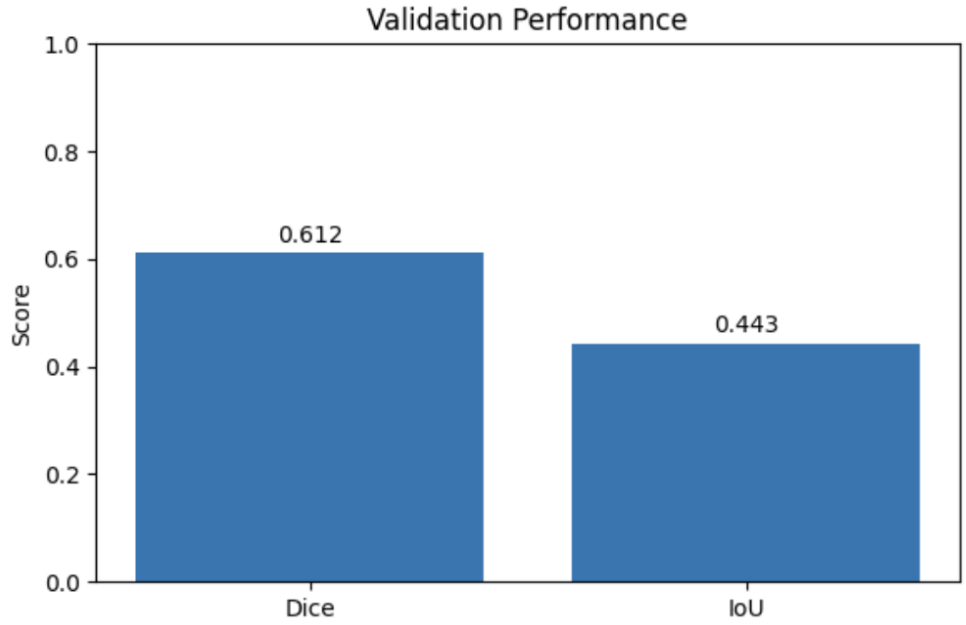


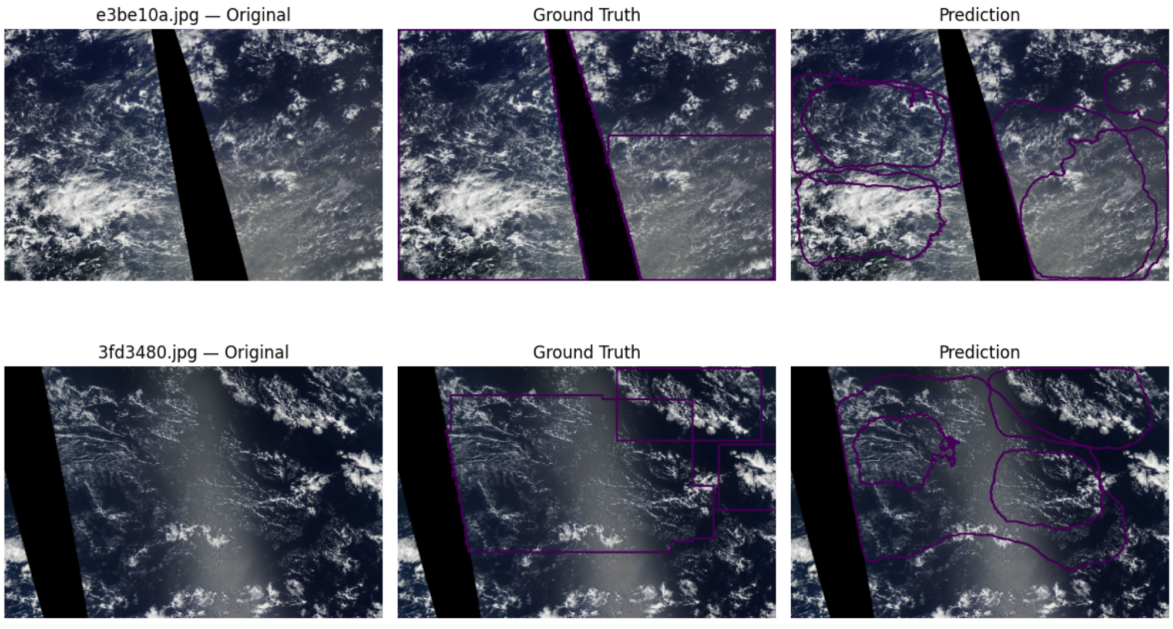
Figure 20: Model 2 Validation Performance of the U-Net with ResNet50

Figure 20 compares the overall Dice score and Mean IoU obtained on the validation set.

4.3.2. Prediction Visualization

To visually evaluate the segmentation results, predictions from the trained model were compared with the original satellite images and the corresponding ground-truth masks.

Four validation images were selected for visualization. For each image, the original satellite image, the ground-truth mask contours, and the predicted mask contours are shown side by side.



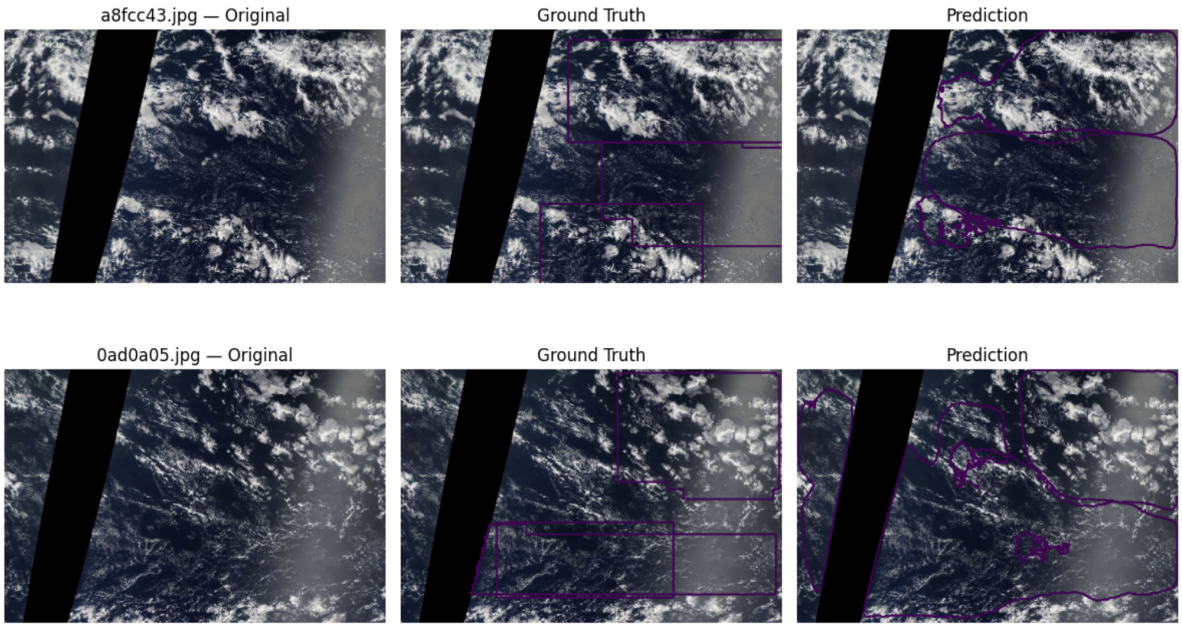


Figure 21: Model 2 comparison of Satellite Images, Ground Truth Masks, and Predictions

The visual comparison shows that the model was able to detect the main cloud regions in the selected validation images. However, differences can be observed between the predicted contours and the ground-truth contours, particularly around the boundaries and coverage of some cloud regions. Some predicted regions are smaller than the corresponding ground-truth regions or do not completely overlap with them.

These observations are consistent with the quantitative evaluation, which shows that segmentation performance differs across cloud categories.

4.3.3. Test Set Prediction

After evaluating the model on the validation set, the selected U-Net with ResNet50 model was used to generate predictions for the test images.

The test set contained 3,698 satellite images. The same preprocessing used for the validation data was applied to the test images, including resizing and ImageNet normalization.

The trained model generated four output channels corresponding to Fish, Flower, Gravel, and Sugar. A sigmoid function was applied to convert the model outputs into probabilities, and a threshold of 0.5 was used to obtain binary segmentation masks.

The predicted masks were then converted back to Run-Length Encoding (RLE) format. A submission file was created containing one prediction for each image and cloud category. The final submission contained 14,792 rows, corresponding to 3,698 test images × 4 cloud categories.

Figure 22 shows predictions generated by the selected U-Net with ResNet50 model on seven test images. For each image, the predicted regions of the four cloud categories are displayed on the original satellite image. The different colors represent the four cloud classes: Fish, Flower, Gravel, and Sugar. These examples provide a visual overview of the model's predictions on previously unseen test images.

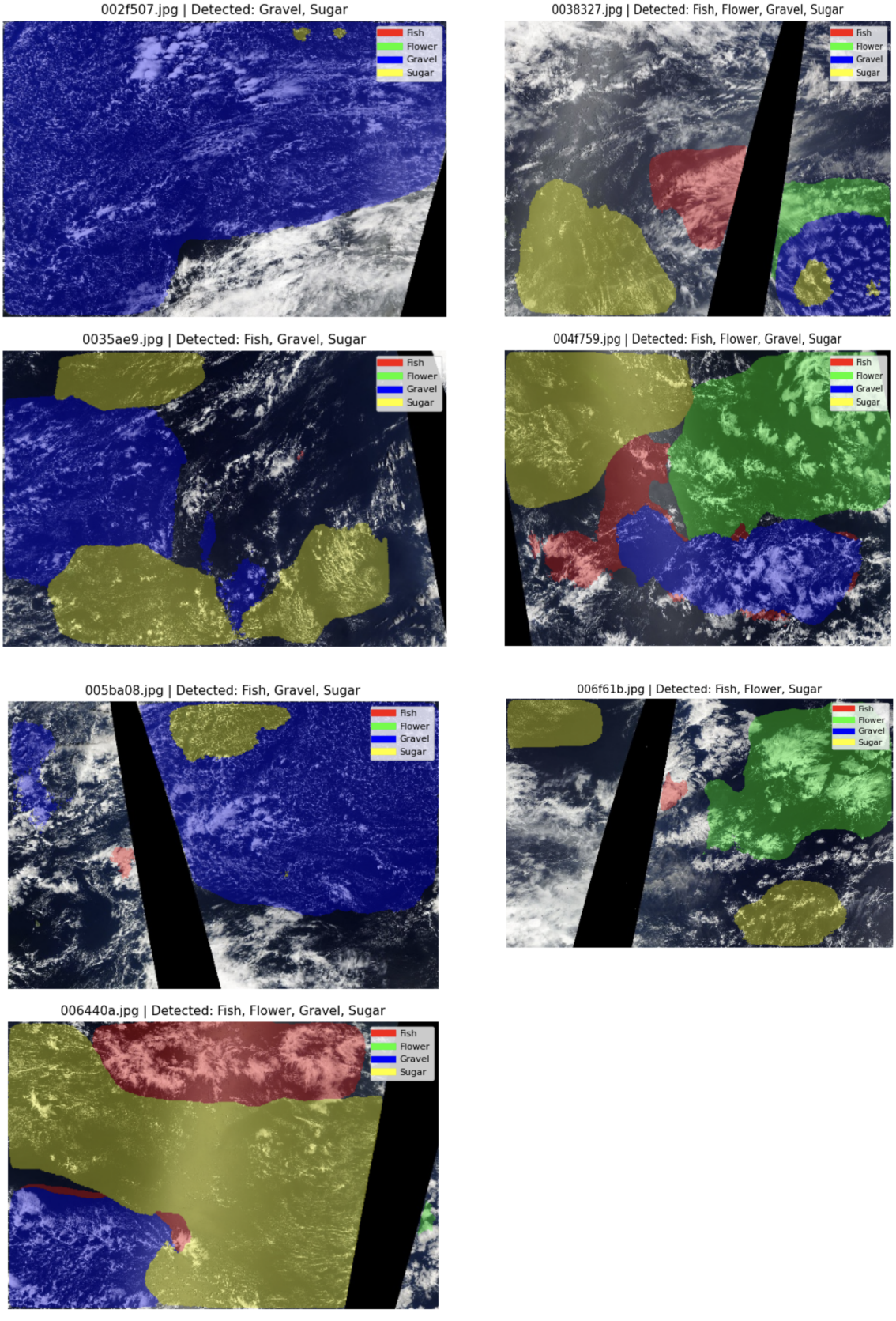


Figure 22: Model 2 Example Test Set Predictions

4.4. Conclusion

The U-Net with ResNet50 model provided a complete encoder-decoder architecture for multi-label cloud segmentation. The ResNet50 encoder, initialized with ImageNet-pretrained weights, extracted hierarchical image features, while the U-Net decoder reconstructed the segmentation maps using encoder-decoder skip connections.

The final validation evaluation produced a Macro Dice score of 0.6118 and a Mean IoU of 0.4432. The per-class results showed differences in segmentation performance, with Flower achieving the highest Dice score and Fish the lowest.

The qualitative validation results further showed that the model was able to identify the main cloud regions, while some inaccuracies remained around cloud boundaries and in the coverage of predicted regions. The trained model was subsequently used to generate predictions for all 3,698 unseen test images and four cloud categories, producing a final submission containing 14,792 predictions.

5. Cloud Segmentation Models 3: Cloud Segmentation Using PSPNet

5.1. Executive Summary

This chapter documents the PSPNet model trained for the cloud-segmentation project: the method, the concrete configuration used, and the validation results. The model predicts four independent masks per image, one for each cloud organisation pattern (Fish, Flower, Gravel, Sugar). Because a pixel can belong to several patterns at once, the task is multi-label rather than multi-class.

Item	Value
Task	Semantic segmentation of cloud patterns in satellite images
Model	PSPNet with a ResNet-34 encoder pre-trained on ImageNet (segmentation_models_pytorch 0.5.0)
Input	384 × 576 pixels, RGB
Output	Four independent masks: Fish, Flower, Gravel, Sugar (multi-label)
Data	5,546 labelled images, split 4,991 train / 555 validation
Framework	PyTorch, Albumentations, OpenCV

Figure 20: Model 3 Project overview.

Training over 25 epochs is stable: training and validation loss decrease together and the validation Dice score rises from 0.5338 after the first epoch to 0.6039 at the end, without a widening train–validation gap. The final model reaches a macro Dice of 0.6039 and a macro IoU of 0.4357 on the 555 held-out validation images. That is close to the U-Net (ResNet-50) model trained on the identical validation split (macro Dice 0.6118), so the two architectures perform on par on this dataset.

5.2. Theoretical Background of the PSPNet Method

PSPNet (Pyramid Scene Parsing Network) is a segmentation architecture built around one central idea: a pixel is easier to classify when the model also knows the global context of the scene it sits in. A plain fully-convolutional network decides each pixel from a limited receptive field, which causes locally plausible but globally inconsistent labels. PSPNet adds a pyramid pooling module on top of the encoder feature map that pools the same features at several spatial scales at once and concatenates the results, so every pixel decision is informed by both local texture and the layout of the whole image.

The trade-off against U-Net is therefore: U-Net recovers spatial detail through skip connections, while PSPNet trades some of that detail for an explicit model of global context.

For the diffuse, large-scale regions in this dataset the second property is the more relevant one, which is why PSPNet was selected as the counterpart architecture.

Component	Role in PSPNet	Role in this project
Encoder (backbone)	A CNN — here ResNet-34 — produces a downsampled feature map with rich semantic content.	Learns cloud texture, brightness and edge structure; ImageNet weights give a usable starting point despite the moderate dataset size.
Pyramid pooling module	Average-pools the encoder features onto 1×1, 2×2, 3×3 and 6×6 grids, reduces each level with a 1×1 convolution, upsamples all levels back and concatenates them with the un-pooled features.	Cloud patterns are large, diffuse and defined by their arrangement over the whole scene, which is exactly the global context this module adds.
Decoder head	A convolution over the concatenated pyramid features, followed by bilinear upsampling back to input resolution.	Produces the four full-resolution mask channels. The head is light — PSPNet has no U-Net-style skip connections, so fine boundaries are less sharp, which matters little for the soft cloud boundaries here.
Output layer	One channel per class with sigmoid activation in the multi-label case.	Four independent probability maps for Fish, Flower, Gravel and Sugar, thresholded at 0.5.

Figure 21: Model 3 Core PSPNet components and their role in this project.

5.3. Application of the PSPNet

5.3.1. Application of PSPNet in this Project

Architecture as instantiated. The model is created from segmentation_models_pytorch as PSPNet(encoder_name="resnet34", encoder_weights="imagenet", classes=4). Scheme 1 shows the resulting forward path with the tensor shapes it actually produces for a 384 × 576 input.

Figure 22 shows the PSPNet architecture as it was instantiated in this project: ResNet-34 encoder (stages 1–3), pyramid pooling module, segmentation head and bilinear upsampling.

One property of this configuration deserves to be stated explicitly, because it shapes the interpretation of every result below. The library default encoder_depth = 3 was not overridden, so only the first three stages of ResNet-34 are part of the forward path: the encoder output is a 48 × 72 feature map with 128 channels (stride 8), and the deeper stages 4 and 5 — which hold roughly 20 of the checkpoint's 21.45 million parameters — are instantiated and stored but never evaluated. Effectively about 1.5 million parameters are trained and used. The model is therefore a PSPNet on a truncated ResNet-34 rather than on the full network. Training and inference use the same setting, so the reported scores are internally consistent; the point is that this was not a deliberate design choice but an unnoticed library default, and that the resulting model is small enough to be the most likely reason the learning curves flatten early.

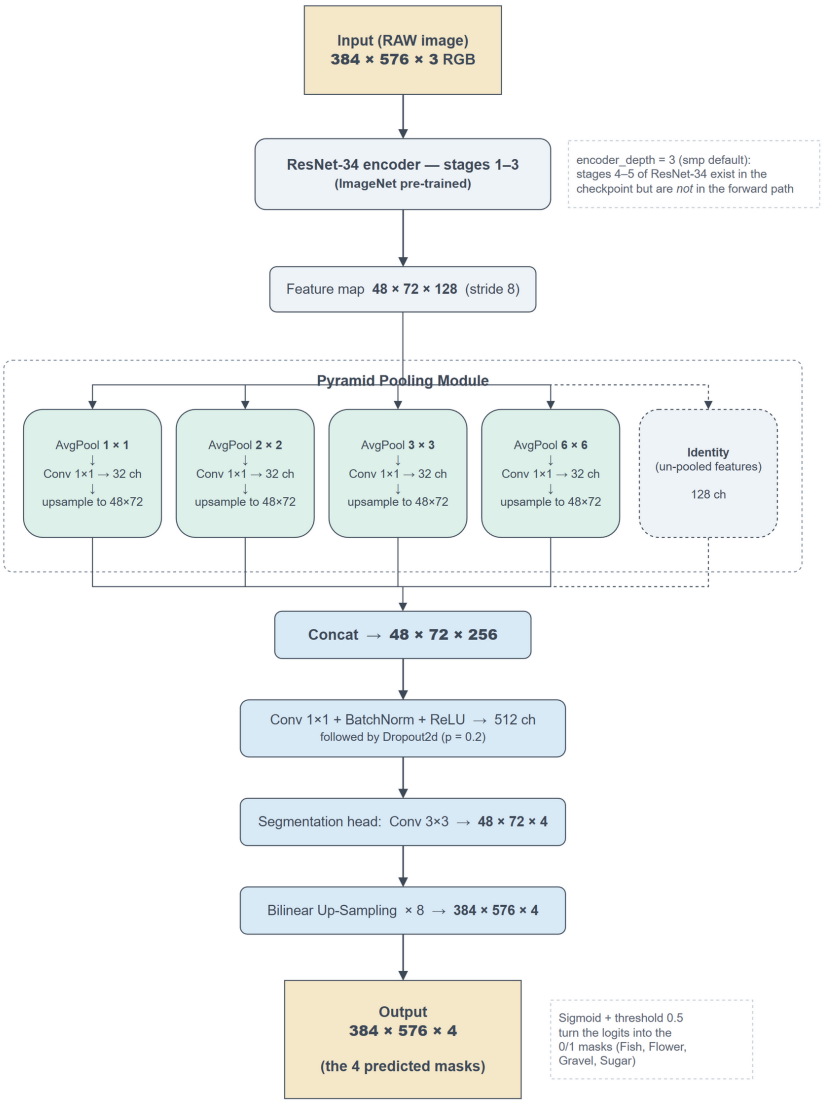


Figure 22: Model 3 instantiated ResNet-34 encoder PSPNet architecture

Training Setup. Images and masks are loaded at 384 × 576 by a custom dataset class that decodes the run-length-encoded masks from train.csv, applies the augmentation pipeline, and normalises the image with the ImageNet statistics the encoder expects.

Setting	Value
Loss	BCEWithLogitsLoss + multi-label Dice loss (weighted 1 : 1)
Optimizer	Adam, learning rate 1e-4, no scheduler
Epochs / batch size	25 / 8
Augmentation (train only)	Horizontal flip (p = 0.3), vertical flip (p = 0.3), random brightness / contrast (p = 0.8), rotation (p = 0.6)
Train / validation split	Random 90 / 10 split on unique image names (random_state = 42), so all four label rows of an image stay on the same side
Parameters	21.45 M stored in the checkpoint, of which ≈ 1.5 M lie in the forward path

Hardware	Single CUDA GPU

Figure 23: Model 3 training configuration

Two details are worth naming. First, the split is performed on unique image names, not on the label rows, which rules out leakage between training and validation. Second, validation data is never augmented, so the reported scores reflect the un-modified images. The split is a plain random split; a stratified split over the label combinations is planned as a separate optimisation step and is deliberately not part of this baseline.

5.3.2. Results and Interpretations

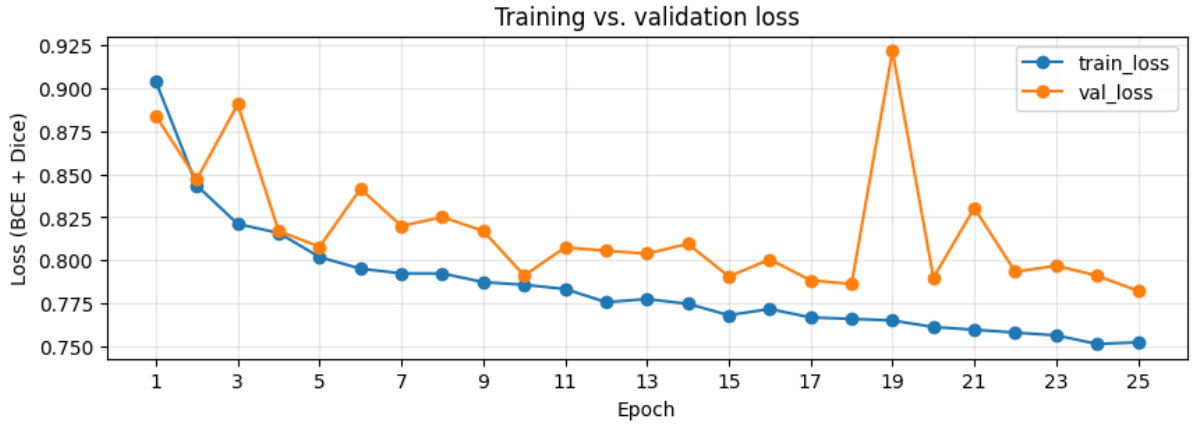


Figure 24: Model 3 training and validation loss (BCE + Dice) over 25 epochs

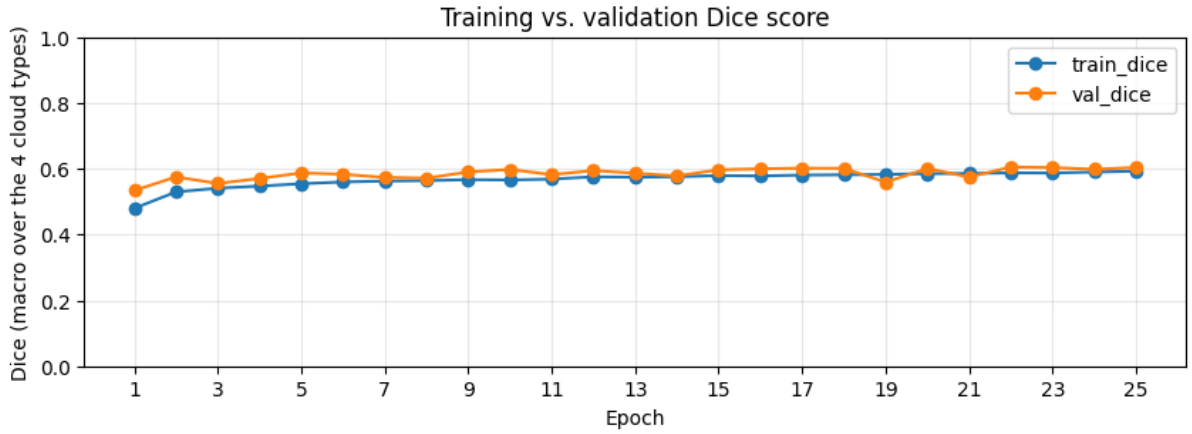


Figure 25: Model 3 training and validation Dice score over 25 epochs

Training loss falls smoothly from 0.9038 to 0.7524. Validation loss falls from 0.8838 to 0.7823, but does so far less evenly: from roughly epoch 4 onwards it sits consistently a little above the training curve and oscillates around a slowly falling trend, with one pronounced outlier at epoch 19 (0.9216, worse than the very first epoch). The training side of that epoch

is entirely undisturbed — its loss step of −0.0010 is an ordinary value for this run, and training Dice even improves slightly — so whatever happened was invisible to the optimisation and affected only the evaluation.

The Dice curves are correspondingly flat. Training Dice rises from 0.4797 to 0.5927 and validation Dice from 0.5338 to 0.6039, with the best single epoch at 0.6053 (epoch 22). Almost all of that gain happens in the first ten epochs; from roughly epoch 10 onwards validation Dice moves within a band of about ±0.02 around 0.60 and the epoch-19 dip is the only visible excursion. The model has converged to what this configuration can reach, and more epochs alone would not help.

Validation Dice sits marginally above training Dice throughout. This is expected here rather than suspicious: the training scores are accumulated from the logits of the update steps, i.e. over a model that is still changing and over augmented images, while the validation scores describe a fixed model on untouched images.

5.3.3. Final validation metrics

The numbers below are produced by re-loading the saved checkpoint and scoring it on the validation loader, on the thresholded 0/1 masks the model actually outputs (threshold 0.5). Macro averages the four per-class scores, so a rare cloud type counts as much as a common one; micro pools the pixels of all four classes and is therefore dominated by the large ones.

Metric	Macro	Micro	Interpretation
Dice coefficient	0.6039	0.6074	Higher is better. Roughly 60 % overlap between predicted and true regions.
IoU	0.4357	0.4362	Higher is better. The stricter of the two overlap measures.
Loss (BCE + Dice)	0.7823	—	Lower is better. The quantity optimised, not a score.

Figure 26: Model 3 final validation metrics of the saved model

Macro and micro scores differ by less than 0.004, which means no single cloud type dominates the pixel count badly enough to distort the aggregate. The per-class table shows where the remaining differences sit.

Cloud type	Dice	IoU
Fish	0.5125	0.3446
Flower	0.6947	0.5322
Gravel	0.5829	0.4114
Sugar	0.6253	0.4548

Mean (macro)	0.6039	0.4357

Figure 27: Model 3 Per-class validation scores

Flower is segmented best and Fish worst, a spread of 0.18 Dice between them considerably larger than any difference between the two architectures compared in section 5.

5.3.4. Predictions on unseen test images

The Kaggle test images carry no ground-truth masks, so they allow a qualitative check only. The trained model is run on the raw image and its output is thresholded into the same mask format the labelled data uses, which makes the existing overlay visualisation applicable to predictions. A cloud type counts as detected when its predicted mask covers at least 500 pixels.

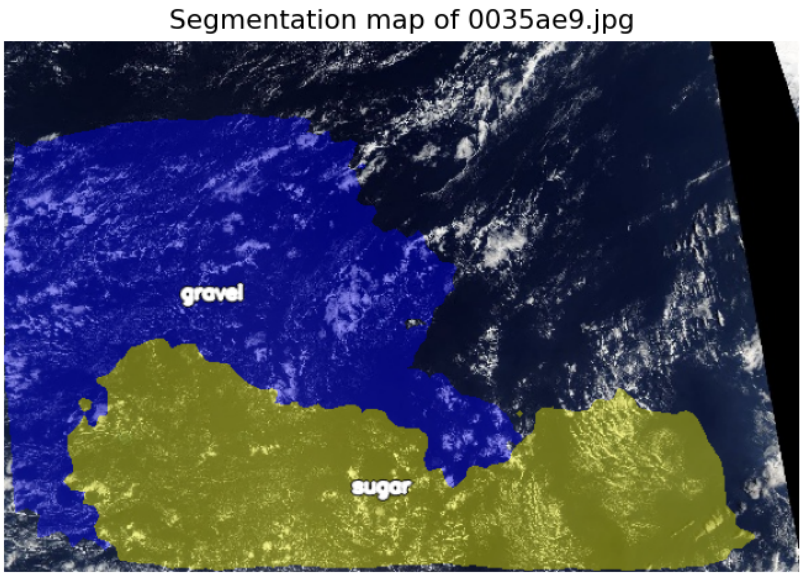


Figure 28: Model 3 segmentation map for test image 0035ae9.jpg

Verdict: the model detected: Gravel and Sugar.

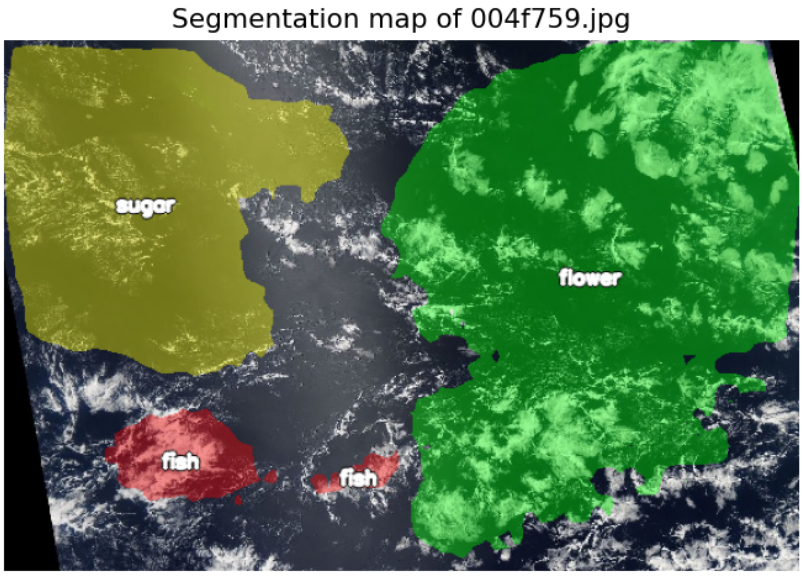


Figure 29: Model 3 segmentation map for test image 004f759.jpg

Verdict: the model detected: Fish, Flower and Sugar.

The predicted regions are large, contiguous and plausibly placed, which is consistent with the pyramid pooling module working as intended: the model commits to one pattern per area rather than scattering labels. Their boundaries are smooth and follow the cloud structure only approximately — the expected consequence of an architecture without skip connections whose output is upsampled by a factor of eight, and a visual explanation of why Dice stalls around 0.60. Figure 4 also shows the Fish weakness from Table 5: the Fish regions are small, fragmented patches, whereas Flower and Sugar are recovered as single coherent areas.

5.4. Conclusion

PSPNet is a suitable architecture for this task: the cloud patterns are large, diffuse regions whose identity depends on the arrangement of the whole scene, which is what the pyramid pooling module models explicitly. Training was stable over 25 epochs with no sign of overfitting, and the model reaches a macro Dice of 0.6039 and a macro IoU of 0.4357 on the held-out validation set — on par with the U-Net (ResNet-50) baseline evaluated on the identical split, and reached with roughly 1.5 million active parameters. This model should be regarded as a baseline, not as a tuned final model.

The following steps are the most promising ones from here, in this order:

- Give the encoder more depth than the library default of 3. The curves flatten after ten epochs and only about 1.5 million parameters are being trained, so capacity is the most plausible limit. Note that encoder_depth and upsampling are coupled: depth 4 requires upsampling 16 and depth 5 requires upsampling 32, otherwise the model silently returns masks smaller than the input. More depth also means a coarser feature map for the pyramid pooling module (24 × 36 or 12 × 18 instead of 48 × 72), so this should be run as

an ablation over the three settings rather than assumed to be an improvement; depth 4 is the natural first attempt.

- Stratified train/validation split over the label combinations, so rare combinations are represented proportionally in both sets — planned as an explicit before/after comparison against this baseline.
- Per-class threshold optimisation on the validation set instead of the global 0.5 cutoff, which mainly targets the weak Fish class.
- Stabilise the batch-normalisation statistics — a larger batch size, or a lower batch-norm momentum — so that the validation score stops depending on which images happen to end an epoch. Section 4.1 traces the epoch-19 spike to this rather than to the optimiser, which makes it the more targeted fix; a learning-rate schedule is still worth adding, but for smoother convergence rather than to remove that outlier.

6. Cloud Segmentation Models Comparison

6.1. Comparison of the Three Segmentation Models

Four model configurations were developed and evaluated during the project: a compact baseline U-Net, a U-Net with a ResNet34 encoder, a U-Net with a ResNet50 encoder, and a PSPNet with a ResNet34 encoder.
The modeling process followed an iterative approach. First, a compact U-Net was developed as a baseline to establish an initial reference for segmentation performance. To investigate the effect of a stronger pretrained encoder while keeping the U-Net architecture, a U-Net with a ResNet34 encoder was then evaluated. This configuration also provided an important reference for the later comparison between U-Net and PSPNet using the same ResNet34 encoder.
The next step was to increase the encoder capacity while retaining the U-Net structure. Therefore, the ResNet34 encoder was replaced with an ImageNet-pretrained ResNet50 encoder. This allowed us to examine whether a deeper and more expressive encoder could improve segmentation performance.
Finally, a different segmentation architecture was investigated using PSPNet with a ResNet34 encoder. Using ResNet34 for both U-Net and PSPNet provided a more direct comparison of the two segmentation architectures, while the comparison with U-Net ResNet50 additionally showed the effect of using a stronger encoder.

The purpose of this progression was to investigate whether the observed segmentation performance could be improved by increasing encoder capacity within U-Net and whether a different segmentation architecture could provide comparable or improved results.

6.2 Validation Performance Comparison

The three models were evaluated using the validation set and the Macro Dice score as the primary comparison metric. The validation split was based on the same set of 555 held-out images for the models included in the final comparison.
Figure 30 presents the per-class Macro Dice results for the PSPNet and U-Net with ResNet50 models.

Cloud type	PSPNet (ResNet-34)	U-Net (ResNet-50)	Difference
Fish	0.5125	0.5478	− 0.0353
Flower	0.6947	0.6962	− 0.0015
Gravel	0.5829	0.5729	+ 0.0100
Sugar	0.6253	0.6301	− 0.0048
Mean (macro)	**0.6039**	**0.6118**	**− 0.0079**

Figure 30: Macro Dice per cloud type, PSPNet vs. U-Net (ResNet-50) comparison

The comparison shows that the U-Net with ResNet50 achieved a Macro Dice score of 0.6118, while the PSPNet with ResNet34 achieved 0.6114. The difference between the two

models is therefore only 0.0004 Dice points, indicating very similar aggregate validation performance.

The per-class results provide additional information that is not visible from the overall score alone. The models show different behavior across the four cloud categories, with the largest difference occurring for the Fish class. This indicates that the overall similarity between the models does not mean that they produce identical segmentation behavior.

It is also important to interpret this comparison in the context of the model configurations. The U-Net with ResNet50 uses a deeper encoder and the U-Net skip-connection-based decoder, whereas the PSPNet uses a ResNet34 encoder and a pyramid-pooling-based segmentation architecture. Therefore, the comparison demonstrates the validation behavior of the specific model configurations used in this project rather than isolating the effect of a single architectural component.

6.3 Comparison of Model Predictions

In addition to the quantitative validation metrics, the three models were applied to the same seven unseen test images for qualitative comparison.

Image	U-Net + ResNet50	U-Net + ResNet34	PSPNet + ResNet34
002f507.jpg	Gravel, Sugar	Gravel, Sugar	Gravel, Sugar
0035ae9.jpg	Gravel, Sugar, Fish	Gravel, Sugar, Fish	Gravel, Sugar
0038327.jpg	Gravel, Sugar, Fish, Flower	Gravel, Sugar , Flower	Sugar, Fish, Flower
004f759.jpg	Gravel, Sugar, Fish, Flower	Sugar, Fish, Flower	Sugar, Fish, Flower
005ba08.jpg	Gravel, Sugar, Fish	Gravel , Fish	Gravel
006440a.jpg	Gravel, Sugar, Fish, Flower	Gravel, Sugar, Fish, Flower	Gravel, Sugar, Fish
006f61b.jpg	Sugar, Fish, Flower	Gravel, Sugar, Fish, Flower	Gravel, Sugar, Fish

Figure 31: Illustrative comparison of model prediction results

Figure 31 presents the cloud categories detected by each model for these images. The predictions show that the models do not produce identical segmentation patterns. In particular, the U-Net with ResNet50 generally identifies a larger number of cloud categories in several of the selected test images.

However, the test images do not contain ground-truth segmentation masks. Therefore, differences in the number of detected cloud categories cannot be interpreted as differences in prediction accuracy. The qualitative comparison is used only to illustrate how the three trained models behave on previously unseen images, while quantitative model comparison is based on the labelled validation set.

6.4 Final Model Comparison

The final comparison was based primarily on the Macro Dice score obtained on the validation set.

Model	Encoder	Best Validation Dice
U-Net	ResNet34	0.5974
U-Net	**ResNet50**	**0.6118**
PSPNet	ResNet34	0.6114

Figure 32: Final comparison of best validation dice results

Figure 32 summarizes the validation results of the three models.

The results show that the U-Net with ResNet50 achieved a Macro Dice score of 0.6118. The PSPNet with ResNet34 achieved a very similar score of 0.6114, while the baseline U-Net with ResNet34 achieved 0.5974.

The results therefore show a clear improvement from the baseline U-Net to the U-Net with the stronger ResNet50 encoder. At the same time, the PSPNet experiment demonstrated that a different segmentation architecture using a ResNet-based encoder could achieve a comparable validation result.

Considering the validation performance together with the complete modelling workflow, the U-Net with ResNet50 was selected as the final model for generating predictions on the unseen test set. It provided the highest Macro Dice score among the evaluated configurations and was subsequently used to generate the final test-set predictions and submission file.

6.5. Recommended Model Selection

The modeling process demonstrates a progressive evaluation of architectural alternatives rather than relying on a single network configuration. The baseline U-Net established the initial segmentation approach, the U-Net with ResNet50 investigated the effect of a substantially stronger pretrain d encoder within the same architecture, and PSPNet provided a comparison with a different segmentation framework while retaining a ResNet-based encoder.

The final validation results identified the U-Net with ResNet50 configuration as the selected model, with a Macro Dice score of 0.6118. The very small difference from PSPNet with ResNet34 (0.6114) also shows that the two final configurations achieved highly similar aggregate validation performance on this dataset.

The selected U-Net with ResNet50 model was therefore used for the final predictions on the 3,698 unseen test images, producing four segmentation outputs per image and a total of 14,792 prediction records for the final submission.

Summary and Conclusion

This report summarizes our Liora group project, which was completed as part of the Liora Data Scientist course. The main objective was to learn and apply data preprocessing, deep learning, and image segmentation techniques in a real-world context using satellite imagery.

The project focused on the segmentation of four different cloud classes: Fish, Flower, Gravel, and Sugar. After exploring and preprocessing the dataset, we developed and evaluated several segmentation models through an iterative modelling process. We first established a compact U-Net as a baseline, then investigated U-Net architectures with ResNet34 and ResNet50 encoders, and finally evaluated a PSPNet with a ResNet34 encoder. This progression allowed us to examine both the effect of encoder capacity and the effect of using a different segmentation architecture.

The models achieved different levels of segmentation performance on the validation data. Based on the final validation comparison, the U-Net with the ResNet50 encoder was selected as the final model for generating predictions on the unseen test images. The selected model was subsequently used to generate the final test-set predictions and submission.

Beyond the final model selection, the project provided practical experience in data preprocessing, model development, training, validation, performance evaluation, and comparison of deep learning architectures. More importantly, it strengthened our ability to apply these techniques to a real-world computer vision problem and provided practical skills and confidence that can be further developed and applied in future professional projects.

References

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and Flexible Image Augmentations. *Information, 11*(2), 125.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Iakubovskii, P. (2019). *Segmentation Models PyTorch*. GitHub. Segmentation Models PyTorch

Kaggle. (2019). *Understanding Clouds from Satellite Images*. Kaggle competition dataset.

Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *International Conference on Learning Representations (ICLR)*.

Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241.

Zhao, H., Shi, J., Qi, X., Wang, X., & Jia, J. (2017). Pyramid Scene Parsing Network. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2881–2890.